# DeepSeek-Inspired Efficiency Under Data-Limited Training: A Controlled Study of MLA, Sparse MoE Routing, V3-Style Load Balancing, and Multi-Token Prediction

Marko Miovski

July 29, 2026

**Abstract**

Architectural efficiency mechanisms can reduce theoretical storage or per-token computation but their realized benefits may depend on scale and optimized systems support. This study evaluates six DeepSeek-inspired decoder variants—a dense control, a Multi-Head Latent Attention (MLA) analogue, multi-token prediction (MTP), sparse mixture-of-experts (MoE), MLA+MoE, and V3-style expert-bias routing—under one controlled local protocol. Each model was trained at nominal budgets of 10M, 25M, and 50M tokens with ten aligned seeds, yielding 180 completed runs. Test next-token cross-entropy was the primary inferential outcome, analyzed with paired contrasts, exact sign-flip tests, percentile-bootstrap intervals, and Holm correction across 21 planned comparisons. Efficiency was measured separately through end-to-end training-token throughput, peak allocated GPU memory, and total-parameter versus activated-parameter exposure. All six models improved as the token budget increased, with mean test loss decreasing by 0.679–0.810 from 10M to 50M tokens and every aligned seed improving. Dense remained close to the best mean test loss while providing the best systems profile, reaching 4,336.0 tokens/s and 2.83 GiB peak allocated memory at 50M tokens. MLA was consistently worse than Dense in quality, throughput, and memory, including a 0.190 higher mean test loss at 50M. MoE activated an estimated 35.7% of its 220.146 million stored parameters and remained quality-competitive but no MoE–Dense test-loss advantage survived the primary correction; its local execution was approximately 30% slower and used 51.6% more peak memory. V3-style routing did not improve quality and ended with higher terminal expert-load variance than ordinary MoE. MTP reduced its auxiliary loss without establishing a reliable next-token sample-efficiency gain. Combining MLA with MoE compounded systems cost without improving quality. Overall, the mechanisms redistributed stored capacity, activated capacity, optimization signal, and execution overhead rather than producing a universal efficiency gain. Under this single-GPU, data-limited regime, Dense offered the strongest overall balance while the expected benefits of sparse routing, MLA, and MTP remained dependent on systems support, inference conditions, or training scale not exercised by the experiment.

## Contents

## 1. Introduction

Large language models have improved as model capacity, training data, and computation have increased but those gains have made efficiency a central design problem. A larger dense Transformer stores more parameters and activates the same feed-forward path for every token. This can improve representational capacity, yet it also increases memory use and computation unless the model is trained and served with sufficient data and hardware [9, 11]. Due to this, recent architectures seek to separate stored capacity from per-token work, compress attention state, control sparse routing, or provide denser supervision during pretraining. DeepSeekMoE, DeepSeek-V2, and DeepSeek-V3 combine several of these ideas at production scale. This includes fine-grained sparse experts, shared experts, Multi-Head Latent Attention (MLA), auxiliary-loss-free load balancing, and multi-token prediction (MTP) [2–4].

These mechanisms are promising but their reported advantages do not transfer automatically to smaller systems. Production gains may depend on long-context inference, large batches, expert parallelism, fused kernels, communication libraries, or training budgets large enough for specialization to emerge. A mechanism can reduce an analytical quantity such as activated parameter count while still increasing wall-clock time or device memory in a local implementation. Similarly, an auxiliary objective can be learned successfully without improving ordinary next-token prediction. This means evaluating such mechanisms under constrained hardware requires more than reproducing their names or reporting a single validation loss. It requires a controlled comparison that measures predictive quality, systems cost, parameter exposure, and mechanism-specific behavior separately.

### 1.1 Motivation: Architectural Efficiency Under Constrained Compute

Efficiency in language modeling is multidimensional. Predictive quality matters but so do the number of parameters that must be stored, the number activated for each token, the memory allocated during training, the rate at which tokens are processed, and the stability of any routing mechanism. These quantities need not move together. Sparse MoE models, for example, can expose each token to only a fraction of total model capacity, yet the complete expert bank must still reside in memory and the routing path introduces dispatch and control overhead [7, 12]. MLA is motivated primarily by reducing the key–value cache during autoregressive inference but its projection and reconstruction operations may remain costly during full-sequence training [3]. MTP supplies additional future-token targets but the extra optimization work does not guarantee lower next-token loss at every model scale or token budget [8].

The amount of training data relative to model capacity further changes how these mechanisms should be interpreted. Scaling-law studies show that parameters and tokens must be considered jointly. Increasing model size without a corresponding increase in training data can leave capacity underexposed [9, 11]. This issue is particularly important for sparse models because total parameters and activated parameters describe different aspects of the architecture. Tokens per total parameter characterize exposure of stored capacity, whereas tokens per activated parameter characterize approximate exposure of the computation used on each token path. Because of this, a local study should report both views rather than treating parameter count as a single undifferentiated quantity.

This study is motivated by the need for that controlled, mechanism-aware evidence. The objective is not to identify the largest or most capable language model that can be trained locally. It is to determine what happens when several DeepSeek-inspired mechanisms are implemented in a common decoder framework, trained under the same short-to-medium token budgets, and evaluated with aligned

experimental units. The constrained setting is useful precisely because it exposes which benefits appear directly from the architecture and which appear to depend on larger-scale systems support.

### 1.2 Study Problem and Scope

The study compares six decoder-only model families: a dense control, an MLA analogue, an MTP model, a sparse MoE model, a combined MLA+MoE model, and a V3-style routing model. Each family was trained at nominal budgets of 10M, 25M, and 50M tokens using ten aligned seeds, producing a balanced matrix of 180 completed runs. All runs used the same FineWeb-Edu-derived corpus sample, tokenizer, context length, optimizer family, and local hardware environment. Test next-token cross-entropy is the primary quality outcome, supplemented by validation loss, perplexity, end-to-end training-token throughput, peak allocated GPU memory, parameter exposure, and terminal routing or MTP diagnostics.

The comparison is intentionally mechanism-focused rather than parameter-matched in every dimension. The dense, MLA, and MTP models differ in total parameter count while the sparse models contain substantially more total capacity but activate a smaller subset per token. The study therefore does not ask which architecture is universally best at a fixed parameter count. It asks how the implemented mechanisms change the joint tradeoff among quality, storage capacity, activated capacity, training cost, and stability under one controlled protocol.

The scope is also narrower than a full DeepSeek reproduction. The implementation excludes distributed expert parallelism, production all-to-all communication, custom sparse kernels, FlashMLA, generation-time KV-cache benchmarking, speculative decoding, instruction tuning, reinforcement learning, and reasoning evaluation. The results consequently describe the local implementations, not the complete DeepSeek-V2, DeepSeek-V3, or DeepSeek-R1 systems [5]. This boundary is essential because a negative local training result does not invalidate a mechanism designed for optimized inference or distributed execution, and a positive local result would not establish frontier-scale superiority.

### 1.3 Research Questions

The central research question is:

> *How do DeepSeek-inspired architectural and training-efficiency techniques affect model quality, memory usage, throughput, activated parameter count, expert utilization, routing stability, and optimization behavior when implemented at small scale under fixed compute constraints?*

The controlled architecture ladder gives this question the six original mechanism-specific forms:

1. What is the control model's quality and efficiency?
2. Does MLA-style compression improve memory or throughput at small scale?
3. Does sparse routing improve activated-parameter efficiency?
4. Do the V2-style components remain stable together?
5. Does routing design improve utilization and stability?

6. Does MTP improve optimization or sample efficiency?

A cross-cutting question concerns the data-limited regime: how do the conclusions change when sparse models are viewed through tokens per total parameter rather than tokens per activated parameter? This distinction is used to interpret capacity and computation rather than to claim a new scaling law. The experiment also does not provide the same-architecture capacity sweep required to identify classical double descent.

### 1.4 Contributions

This study makes four contributions. First, it provides a balanced, multi-seed comparison of six related model families across three token budgets. The use of ten aligned seeds permits paired comparisons and avoids treating the 180 runs as independent observations. The design is broad enough to compare the major mechanisms while retaining a common training and evaluation protocol.

Second, the study evaluates efficiency through several distinct quantities rather than through predictive loss or parameter count alone. It reports total, trainable, and activated parameters; token exposure under each definition; end-to-end training-token throughput; peak allocated GPU memory; and mechanism-specific routing and MTP diagnostics. This separates theoretical sparse-capacity advantages from realized local systems behavior.

Third, the analysis uses a prespecified paired statistical framework. Test loss is treated as the primary inferential outcome, with repeated-measures global tests, exact paired sign-flip tests, paired bootstrap intervals, standardized paired effects, and Holm correction across the complete 21-contrast test-loss family.

Fourth, the study provides a mechanism-first interpretation of the results. It connects observed differences to the architectural operations that could plausibly produce them while distinguishing supported explanations from causal proof. This is important because the main empirical lesson is not that one mechanism dominates every metric. Rather, the mechanisms redistribute capacity, optimization signal, memory demand, and execution overhead in different ways. Additionally, their value depends on which resource constraint is being optimized. The resulting evidence is intended to clarify those tradeoffs within a constrained, data-limited training regime.

## 2. Background and Related Work

The mechanisms studied here come from a broader effort to increase language-model capacity without allowing training and inference cost to grow at the same rate. Prior work has approached that problem through attention compression, conditional computation, routing control, and denser prediction objectives. These mechanisms were originally developed and validated in substantially larger systems, often alongside specialized kernels, distributed execution, and long training runs. The purpose of this section is to explain the intended role of each mechanism and the tradeoff it is designed to address.

### 2.1 Dense Decoder-Only Transformers as the Control

The Transformer replaced recurrence with self-attention and position-wise feed-forward layers, allowing each token representation to combine information from the preceding context through learned attention weights [17]. Decoder-only language models apply this architecture autoregressively: given tokens $x_{<t}$,

the model estimates $p_\theta(x_t \mid x_{<t})$ and is trained by next-token cross-entropy. This objective became the standard pretraining formulation for modern generative language models [1].

In a dense Transformer, the same attention and feed-forward parameters are available to every token. This means total, trainable, and per-token activated parameter counts coincide when all parameters are trainable. Dense execution is comparatively simple because it maps well to large matrix multiplications, avoids routing decisions, and provides a clear relationship between stored capacity and work performed for each token. It is consequently the appropriate control for determining whether a more specialized mechanism improves quality, memory use, throughput, or parameter exposure rather than merely adding architectural complexity.

Prior scaling studies show that language-model loss depends jointly on model size, training data, and compute rather than on parameter count alone [9, 11]. A larger model can be more sample-efficient but it can also remain undertrained when the token budget does not grow with capacity. This interaction motivates holding the training-token budget fixed while reporting both predictive quality and systems cost.

### 2.2 Multi-Head Latent Attention

Standard multi-head attention forms separate key and value representations for each layer and attention head. During autoregressive generation, previously computed keys and values are retained in a key–value (KV) cache so that the model does not recompute the full prefix at every decoding step. The cache grows with sequence length, layer count, and key/value dimensionality, thereby making memory capacity and memory bandwidth important inference constraints.

DeepSeek-V2 introduced Multi-Head Latent Attention (MLA) to reduce this cost [3]. Rather than storing full head-specific key and value states, MLA compresses them into a lower-dimensional latent representation from which the required attention states can be recovered. The central efficiency claim is consequently tied to KV-cache compression during autoregressive inference. It does not imply that every MLA implementation must reduce full-sequence training memory or improve training throughput; additional projection and reconstruction operations can remain visible during training, especially without fused kernels or a cache-aware decoding path.

The current study implements an MLA-style latent projection within the same decoder framework as the dense control. This preserves the main architectural idea—compressing the attention representation before reconstructing head-specific states—while treating production KV-cache management, FlashMLA, and kernel-level optimization as out of scope. The relevant empirical question is correspondingly narrow and asks whether this local analogue improves measured training quality or systems behavior under the selected scale and hardware.

### 2.3 Sparse Mixture-of-Experts Models

Mixture-of-Experts (MoE) models increase total capacity through conditional computation. In a Transformer MoE layer, a router assigns each token to a small subset of feed-forward experts and only those selected experts process that token. This indicates the model can store substantially more parameters than it activates on a single token path. GShard and Switch Transformers demonstrated that this separation can support very large models while also identifying routing imbalance, communication cost, and training instability as central engineering problems [7, 12].

DeepSeekMoE extends this design with two ideas intended to improve expert specialization: finer

expert segmentation and the isolation of shared experts [2]. Finer segmentation permits a token to combine a more flexible set of narrower routed experts while shared experts capture information that may otherwise be redundantly learned across routed experts. DeepSeek-V2 combines this sparse feed-forward design with MLA, framing the model in terms of a large total parameter count but a much smaller number of parameters activated for each token [3].

This distinction is essential but incomplete. Fewer activated parameters do not automatically imply lower wall-clock time or lower device memory. Expert weights must still be stored, routing and token dispatch introduce overhead, and efficient sparse execution may depend on large batches, expert parallelism, communication primitives, and specialized kernels. For this reason, the study treats total capacity, activated capacity, throughput, and peak memory as separate measurements rather than using activated parameter count as a proxy for realized speed.

### 2.4 Routing Balance and V3-Style Expert-Bias Control

Sparse models require the router to distribute tokens across experts without allowing a small subset to absorb most assignments. Conventional MoE systems commonly add an auxiliary load-balancing term to the language-model objective. This supplies direct gradient pressure toward more even routing but it also changes the optimization target because a sufficiently strong balancing term can compete with the representations preferred by next-token prediction.

DeepSeek-V3 introduced an auxiliary-loss-free balancing strategy intended to separate load control from the main model objective [4]. Instead of adding the principal balancing signal to the loss, the router maintains expert-specific selection biases that are adjusted according to observed expert load. This implies the logits used to choose experts are shifted toward underused experts and away from overused experts while the original affinity scores continue to determine the routing weights.

The design creates a direct comparison between two forms of control. Auxiliary-loss routing modifies optimization through backpropagation; expert-bias routing modifies discrete expert selection through an external update rule. Their behavior cannot be judged from language-model loss alone. Routing entropy describes how dispersed the router probabilities are while expert-load variance describes how evenly top-$k$ selections are distributed. Neither statistic independently proves useful specialization because concentrated routing may indicate either meaningful specialization or collapse, and uniform routing may reflect balance without differentiated expert function. For that reason, the study evaluates quality and terminal routing behavior together.

### 2.5 Multi-Token Prediction

Standard autoregressive pretraining supervises one next-token target at each position. Multi-token prediction (MTP) augments that objective by asking a shared model representation to predict several future tokens through separate prediction heads. The motivation is to provide a denser learning signal and encourage representations that remain useful beyond the immediately following token. Prior work reports that the benefit can increase with model scale and can be especially pronounced on generative tasks such as code while also creating opportunities for faster inference when multiple proposed tokens are verified together [8].

DeepSeek-V3 includes MTP as a pretraining objective alongside its sparse architecture and routing changes [4]. Those large-scale results do not establish that the same objective will improve a smaller model trained for a short token budget. Auxiliary heads add parameters and optimization work, and

success on their future-token losses need not transfer to lower ordinary next-token loss.

The local implementation uses two auxiliary future-token heads on top of the shared decoder trunk. The evaluation remains next-token cross-entropy, which permits a controlled comparison with the dense baseline. Generation-time verification, speculative decoding, instruction tuning, and reasoning evaluation are excluded. This separation is important because an MTP model may learn its auxiliary task without demonstrating either better next-token quality or faster decoding in the tested system.

### 2.6 Data-Limited and Overparameterized Training

Empirical scaling laws describe language-model loss as a function of parameters, data, and compute [11]. Later compute-optimal analysis showed that increasing parameter count without a corresponding increase in training tokens can leave large models undertrained, and that model size and data should be scaled together when compute is allocated optimally [9]. These findings make the capacity-to-data ratio central to any comparison performed under fixed token budgets.

The interpretation is more complicated for sparse models. Total parameters describe stored representational capacity while activated parameters describe the subset used on a token path. A sparse model may receive relatively few tokens per total parameter but more tokens per activated parameter than a smaller dense model. Trainable parameters form a third concept when some stored parameters are frozen, although all parameters were trainable in the present experiment. Reporting these quantities separately avoids treating a single parameter count as a complete measure of either optimization exposure or computational work.

The term *overparameterized* is used here in this capacity-to-data sense, not as a direct claim about a classical interpolation threshold. Language-model tokens are dependent, contexts overlap, and the effective number of examples is not equivalent to the raw token count. Sparse routing further prevents a simple one-to-one analogy between total parameters and the parameters updated by each token. Likewise, classical double descent cannot be established without a sufficiently dense same-architecture capacity sweep.

Most prior evidence for MLA, DeepSeekMoE, auxiliary-loss-free routing, and MTP comes from large systems in which several mechanisms and systems optimizations operate together [3, 4]. That evidence demonstrates feasibility at scale but makes it difficult to determine how each component behaves under constrained local training. This study addresses that narrower gap by introducing the mechanisms progressively, holding the corpus, tokenizer, context length, optimizer, token budgets, seed set, and evaluation protocol fixed while also measuring quality alongside throughput, memory, parameter exposure, and routing diagnostics.

## 3. Methods and Experimental Design

### 3.1 Study Design and Experiment Matrix

**Table 1:** Canonical experimental protocol. The design was balanced across six models, three token budgets, and ten aligned seeds. Actual token counts exceed the nominal budgets by at most 960 tokens because each optimizer step processes 1,024 targets.

| **Component** | **Canonical setting** |
|---|---|
| Experiment matrix | Six models × three budgets × ten aligned seeds = 180 completed runs |
| Token budgets | 10M, 25M, and 50M nominal training target tokens |
| Seeds | 1337, 2027, 4441, 5501, 6173, 8191, 10007, 11213, 12721, 31415 |
| Corpus | FineWeb-Edu `sample-10BT`; 50,000 train, 1,000 validation, and 1,000 test documents |
| Tokenized data | 61,108,958 train; 1,108,976 validation; 1,203,792 test tokens |
| Tokenizer | Byte-level BPE, vocabulary 10,000, minimum frequency 2, trained on retained training text |
| Common model geometry | 12 layers, 12 heads, width 768, context 256, dropout 0.1, RMSNorm, RoPE, tied embeddings |
| Optimization | AdamW; learning rate $3 \times 10^{-4}$; weight decay 0.1; betas $(0.9, 0.95)$; gradient clipping 1.0; fixed learning rate |
| Batch and precision | Four sequences × 256 targets = 1,024 target tokens per step; fp32 |
| Hardware and software | NVIDIA GeForce RTX 4050 GPU; Windows 10; Python 3.11.9; PyTorch 2.6.0+cu124; CUDA 12.4 |
| **Budget execution** | **Steps / actual tokens / overshoot / evaluation batches** |
| 10M | 9,766 / 10,000,384 / 384 (0.00384%) / 50 |
| 25M | 24,415 / 25,000,960 / 960 (0.00384%) / 75 |
| 50M | 48,829 / 50,000,896 / 896 (0.00179%) / 100 |

The seed is the repeated experimental unit. Training windows were randomly sampled with replacement; evaluation used deterministic, overlapping stride-one windows with budget-dependent nested prefixes.

The study used a balanced factorial design to compare six decoder-only language-model variants under the same local training environment. The model families were Dense, MLA, MTP, MoE, MLA+MoE, and V3 Routing. Each model was trained at nominal budgets of 10, 25, and 50 million target tokens using the same ten seeds: 1337, 2027, 4441, 5501, 6173, 8191, 10007, 11213, 12721, and 31415. The resulting matrix contained $6 \times 3 \times 10 = 180$ completed runs.

The aligned seeds make the design paired. For a given budget, models with the same seed were initialized and trained under the same seed-controlled sampling conditions. The seed, rather than an individual token or evaluation window, is the repeated experimental unit. This pairing was retained in the omnibus tests, planned contrasts, confidence intervals for paired differences, and seed-level win counts. The design supports controlled comparisons among the implemented mechanisms but it does not isolate every architectural component independently because the sparse models necessarily differ from the dense models in both total capacity and feed-forward structure.

### 3.2 Data, Tokenizer, and Sequence Construction

The corpus was drawn from the `sample-10BT` configuration of FineWeb-Edu [15]. A single streaming training split was shuffled with a buffer of 10,000 records and seed 1337. The shuffled stream was then consumed sequentially to create nonoverlapping local artifacts containing 1,000 validation documents, 1,000 test documents, and 50,000 training documents. Newlines were normalized, leading and trailing whitespace was removed, and empty records were discarded. The text supplied to tokenizer training contained 236,369,471 characters, including the retained document separators.

A byte-level byte-pair-encoding tokenizer was trained only on the retained training text, with a requested and realized vocabulary size of 10,000 and minimum token frequency of two. Tokenization produced flat `int32` memory-mapped arrays containing 61,108,958 training tokens, 1,108,976 validation tokens, and 1,203,792 test tokens.

All models used a context length of 256. During training, each example consisted of 256 input tokens and the following 256 next-token targets. Start positions were sampled uniformly with replacement from the training memmap. Training windows could consequently overlap or repeat and the processed tokens should not be interpreted as independent observations.

Validation and test evaluation used deterministic stride-one windows from the beginning of each held-out token array. Each evaluation batch contained four windows. The 10M, 25M, and 50M protocols evaluated 50, 75, and 100 batches, respectively, corresponding to 200, 300, and 400 heavily overlapping windows. These windows contributed 51,200, 76,800, and 102,400 target-token loss terms while spanning 455, 555, and 655 distinct target positions. This means the evaluated prefixes were nested but not identical across budgets. Within a budget, every model and aligned seed used the same evaluation sample; cross-budget trends are interpreted with the differing prefix lengths in mind.

### 3.3 Controlled Model Family

All six variants used the same decoder depth, width, vocabulary, context length, dropout, normalization, positional encoding, and tied input/output embeddings. The common backbone contained 12 decoder layers, 12 attention heads, model width 768, dropout 0.1, RMS normalization, rotary positional encoding [16], and tied token-embedding and language-model-head weights. Dense feed-forward blocks used a SwiGLU hidden width of 3,072. The experiment changed only the attention, feed-forward routing, or auxiliary-objective path required by each comparison.

**Table 2:** Controlled model family and parameter accounting. Activated parameters include all non-routed parameters, the shared experts, and the average top-2 routed-expert path in each MoE layer. They are an architecture-level exposure estimate, not measured FLOPs.

| Model | Controlled mechanism | Total params. | Activated params. | Act./total |
|---|---|---|---|---|
| Dense | Dense attention and dense SwiGLU feed-forward blocks | 120,945,408 | 120,945,408 | 1.000 |
| MLA | Local MLA attention analogue; dense SwiGLU feed-forward blocks | 137,460,480 | 137,460,480 | 1.000 |
| MTP | Dense backbone with two separate auxiliary future-token heads | 136,305,408 | 136,305,408 | 1.000 |
| MoE | Dense attention; 12 routed and one shared expert per layer; top-2 routing | 220,146,432 | 78,588,672 | 0.357 |
| MLA+MoE | Local MLA attention analogue combined with the auxiliary-loss MoE path | 236,661,504 | 95,103,744 | 0.402 |
| V3 Routing | MoE geometry with auxiliary-loss-free expert-bias selection control | 220,146,432 | 78,588,672 | 0.357 |

All parameters were trainable, so total and trainable parameter counts were identical. Dense feed-forward width was 3,072. Each routed or shared expert used width 512. MoE variants used 12 routed experts, one shared expert, normalized softmax top-2 weights, and no token dropping. MLA used key–value latent dimension 192, rotary-query dimension 64, non-positional query/key dimension 128 per head, and value dimension 128 per head. V3 Routing updated its non-gradient expert bias at rate 0.001 after each training forward and clipped it to $[-1, 1]$.

The Dense model served as the reference architecture. MLA replaced dense attention with a local Multi-Head Latent Attention analogue inspired by DeepSeek-V2 [3]. It used a 192-dimensional key–value latent projection, a 64-dimensional rotary query component, 128 non-positional query/key dimensions per head, and 128 value dimensions per head. This implementation was intended to test the local behavior of latent attention projections; it did not reproduce the optimized cache, kernel, or deployment stack of the original system.

MTP retained the dense backbone and added two separate auxiliary output heads. The heads predicted tokens one and two positions ahead from the decoder hidden states while the ordinary language-model head remained separate. This local design was motivated by multi-token prediction [8]; it did not implement shared-head MTP, generation-time verification, or speculative decoding.

MoE replaced each dense feed-forward block with a local DeepSeekMoE-style layer [2]. Each layer contained 12 routed experts and one shared expert. The router selected two routed experts per token using softmax scores, normalized the selected weights, and never dropped tokens. Every expert used hidden width 512. MoE and MLA+MoE used a differentiable load-balancing auxiliary loss with weight 0.01. MLA+MoE combined the same MLA attention analogue with the same auxiliary-loss MoE feed-forward path.

V3 Routing used the MoE architecture and expert geometry but removed the differentiable routing auxiliary loss. Instead, it used a non-gradient expert-selection bias inspired by the auxiliary-loss-free strategy described for DeepSeek-V3 [4]. After each training forward pass, the bias was updated toward

uniform expert load at rate 0.001 and clipped to $[-1, 1]$. This controller affected expert selection but did not change the router probabilities used to weight selected expert outputs.

### 3.4 Training Objectives and Optimization Protocol

For a set of $N$ target tokens with contexts $c_j$ and targets $y_j$, ordinary next-token cross-entropy was

$$\widehat{L}_{\mathrm{NTP}} = -\frac{1}{N}\sum_{j=1}^{N} \log p_\theta(y_j \mid c_j). \tag{1}$$

Dense and MLA optimized only this objective. For MoE and MLA+MoE, the training loss also included the sum of the 12 layer-level load-balancing losses,

$$L_{\mathrm{train}} = L_{\mathrm{NTP}} + \sum_{\ell=1}^{12} L_{\mathrm{aux},\ell}, \qquad L_{\mathrm{aux},\ell} = \lambda_{\mathrm{aux}} E \sum_{e=1}^{E} f_{\ell e}\, \bar{p}_{\ell e}, \tag{2}$$

where $E = 12$ routed experts, $f_{\ell e}$ is expert $e$'s fraction of top-$k$ selections in layer $\ell$, $\bar{p}_{\ell e}$ is its mean router probability, and $\lambda_{\mathrm{aux}} = 0.01$. V3 Routing set this auxiliary term to zero and adjusted its expert-selection biases outside gradient descent.

The MTP model optimized

$$L_{\mathrm{train}} = L_{\mathrm{NTP}} + 0.3\left(\frac{L_1 + L_2}{2}\right), \tag{3}$$

where $L_1$ and $L_2$ are the cross-entropies from the separate one-position-ahead and two-position-ahead auxiliary heads. Because the stored training loss combines different terms for ordinary, MoE, and MTP models, combined training loss is not used for cross-model quality comparisons. Validation and test next-token loss from Equation 1 provide the common quality measure.

All runs used AdamW [13] with learning rate $3 \times 10^{-4}$, weight decay 0.1, momentum coefficients $(0.9, 0.95)$, gradient-norm clipping at 1.0, and no learning-rate schedule. Training used full 32-bit floating-point precision. Each step processed four sequences of 256 target tokens, or 1,024 target tokens per optimizer update. The 10M, 25M, and 50M budgets required 9,766, 24,415, and 48,829 steps and terminated after 10,000,384, 25,000,960, and 50,000,896 observed target tokens. The resulting overshoots were 384, 960, and 896 tokens, all below 0.004% of the requested budget.

Periodic validation occurred every 1,000, 2,500, or 5,000 steps for the 10M, 25M, or 50M protocol. Logging occurred every 100, 250, or 500 steps, respectively. Each run concluded with final validation and test evaluation. Checkpoint intervals were disabled; the protocol retained final summaries and logs but did not provide full interruption-and-resumption support.

### 3.5 Evaluation Metrics and Parameter Exposure

#### 3.5.1 Predictive Quality

Validation and test loss were computed as target-token-weighted means of next-token cross-entropy over the requested evaluation batches. Perplexity was derived as

$$\mathrm{PPL} = \mathrm{e}^{\widehat{L}_{\mathrm{NTP}}}. \tag{4}$$

Lower values indicate better next-token prediction. Test loss was designated as the primary inferential outcome. Perplexity is a monotonic transformation of loss and was retained for interpretation rather than treated as an independent primary endpoint.

### 3.5.2 Parameter Accounting and Exposure

The study separated total, trainable, and activated parameters. Total parameters represent stored model capacity. Trainable parameters represent the parameters exposed to optimization. Activated parameters approximate the parameters used for one token's forward path: all non-routed parameters plus the shared experts and the average top-$k$ routed experts in each MoE layer. This analytical quantity is not a measurement of floating-point operations.

For a run processing $T$ training target tokens, the three exposure ratios were

$$E_{\text{total}} = \frac{T}{P_{\text{total}}}, \qquad E_{\text{trainable}} = \frac{T}{P_{\text{trainable}}}, \qquad E_{\text{activated}} = \frac{T}{P_{\text{activated}}}. \tag{5}$$

They are interpreted as storage/capacity exposure, optimization exposure, and approximate per-token compute exposure, respectively. All parameters required gradients in the final matrix so total and trainable exposure were numerically identical. Activated exposure remained distinct for the sparse models because only two routed experts and the shared expert were active per token in each MoE layer.

### 3.5.3 Systems Metrics

Training-token throughput over measured run time was defined as

$$v_{\text{run}} = \frac{T_{\text{train}}}{\Delta t_{\text{run}}}. \tag{6}$$

The numerator counts training target tokens only. The timer started immediately before the training loop and continued through training, logging, periodic validation, final validation, and final test evaluation. Model construction and data-loader setup occurred before the timer. This means the metric serves as an end-to-end run-rate measure, not pure optimizer-step throughput.

Peak memory was the maximum CUDA tensor memory allocated by PyTorch during the measured run, obtained from `torch.cuda.max_memory_allocated` [14]. It does not include PyTorch's unallocated reserved cache, other device processes, or host-system memory.

### 3.5.4 Routing and MTP Diagnostics

For router probabilities $p_{ie}$ over $E$ routed experts, mean routing entropy was

$$H_{\text{route}} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{e=1}^{E} p_{ie}\log p_{ie}. \tag{7}$$

For realized top-$k$ expert-selection fractions $f_e$, expert-load variance was

$$V_{\text{load}} = \frac{1}{E}\sum_{e=1}^{E}(f_e - \bar{f})^2. \tag{8}$$

Entropy describes the dispersion of router probabilities, whereas load variance describes the balance of realized expert selections. Neither quantity alone establishes useful expert specialization.

The retained routing statistics are latest-forward snapshots. For each run, the final values were taken after test evaluation, averaged across the 12 MoE layers, and then summarized across seeds. They are not averages over the full training trajectory. The reported MTP auxiliary loss is similarly the final optimizer-step value averaged across its two horizons; it is a terminal diagnostic rather than an optimization curve.

### 3.6 Statistical Analysis

#### 3.6.1 Descriptive Summaries

For each model–budget cell, metrics were summarized across the ten seeds using the arithmetic mean and a two-sided 95% Student-$t$ confidence interval,

$$\bar{y} \;\pm\; t_{0.975,9}\frac{s}{\sqrt{10}}, \qquad t_{0.975,9} = 2.262157. \tag{9}$$

The seed is the experimental unit. Tokens, batches, and overlapping evaluation windows were not treated as independent replicates.

#### 3.6.2 Omnibus Model Comparisons

Within each budget, a one-factor repeated-measures analysis of variance tested whether the mean outcome differed across the six models, with seed treated as the block. A Friedman rank test provided a nonparametric companion. For both statistics, 20,000 random permutations were formed by relabeling the six model outcomes within each seed while preserving the seed blocks. If $r$ permuted statistics were at least as extreme as the observed statistic, the reported permutation value was

$$p_{\text{perm}} = \frac{r+1}{20{,}000+1}. \tag{10}$$

These omnibus tests establish whether model choice affects the outcome before interpreting specific mechanism comparisons.

#### 3.6.3 Planned Paired Contrasts

Seven comparisons were specified to match the controlled mechanism questions: MLA−Dense, MTP−Dense, MoE−Dense, MLA+MoE−Dense, MLA+MoE−MLA, MLA+MoE−MoE, and V3 Routing−MoE. For aligned seed $i$,

$$d_i = Y_{i,B} - Y_{i,A}, \qquad \bar{d} = \frac{1}{10}\sum_{i=1}^{10} d_i. \tag{11}$$

For loss, $\bar{d} < 0$ favors Model B. Each contrast reports the mean paired difference, the number of seeds in which Model B performed better, a 20,000-resample paired percentile-bootstrap 95% interval [6], and the standardized paired effect

$$d_z = \frac{\bar{d}}{s_d}, \tag{12}$$

where $s_d$ is the sample standard deviation of the ten paired differences. An exact two-sided sign-flip test enumerated all $2^{10} = 1,024$ assignments of signs to the paired differences. Its minimum attainable nonzero two-sided $p$-value was $2/2^{10} = 0.001953125$.

#### 3.6.4 Multiplicity Control

Test loss was the primary inferential endpoint. Principal significance claims use Holm's sequential correction [10] across the complete family of 21 planned test-loss comparisons. This family was chosen to control the full primary mechanism question set rather than selecting a budget after inspecting the results. Budget-specific corrections and broader metric-group corrections were retained as supplementary sensitivity analyses. Perplexity was not included as an additional primary family because it is determined directly by test loss.

### 3.7 Hardware, Reproducibility, and Implementation Boundaries

All 180 final runs were executed in full 32-bit precision on one NVIDIA GeForce RTX 4050 GPU under Windows 10, Python 3.11.9, PyTorch 2.6.0 with CUDA 12.4, and zero data-loader workers. The final evidence matrix, full-precision statistical outputs, plotting scripts, data and tokenizer provenance, and report-facing artifacts are linked to repository commit `8923ccb2536ccae11a82de9b78838e1b6b900638`. The repository is available at https://github.com/mmiovski/deepseek-reimplementation.

A post-run audit verified the balanced 180-run matrix, canonical configuration closure, artifact provenance, metric and figure hashes, and the tracked analysis outputs. The repository passed 257 automated tests together with formatting, linting, static type, dependency-consistency, whitespace, and evidence-integrity checks. These controls support protocol-level reproducibility but seeded GPU execution is not guaranteed to be bitwise identical across hardware, operating systems, CUDA versions, or library versions.

The implemented models are local DeepSeek-inspired analogues rather than full reproductions of DeepSeek-V2, DeepSeek-V3, or DeepSeek-R1. The study does not include distributed expert parallelism, expert-capacity enforcement, token dropping, custom sparse kernels, production cache compression, FP8 training, speculative decoding, instruction tuning, reinforcement learning, or reasoning evaluation. These boundaries keep the experiment focused on mechanism behavior under constrained local compute; their consequences for external validity are addressed in the limitations section.

## 4. Results and Discussion

This section answers the study questions through the prespecified paired comparisons. Test loss is the primary inferential outcome. Unless stated otherwise, reported significance decisions use the Holm adjustment across all 21 planned test-loss contrasts. For each contrast, the reported difference is Model B minus Model A so a negative value favors Model B. Mechanistic explanations are presented as interpretations of the observed patterns rather than causal proof.

### 4.1 Overall Quality and Token-Budget Scaling

All six models improved as the training-token budget increased. Figure 1 shows a consistent downward shift in mean test loss from 10M to 25M and again from 25M to 50M tokens. From 10M to 50M,

mean test loss decreased by 0.679–0.810 across the six models, and every model improved for all ten aligned seeds. The exact sign-flip $p$-value for each 10M-to-50M paired change was $p = 0.001953$. This suggests the result is not driven by a few favorable seeds. Additional training tokens produced the only improvement that was universal across architectures and replications.

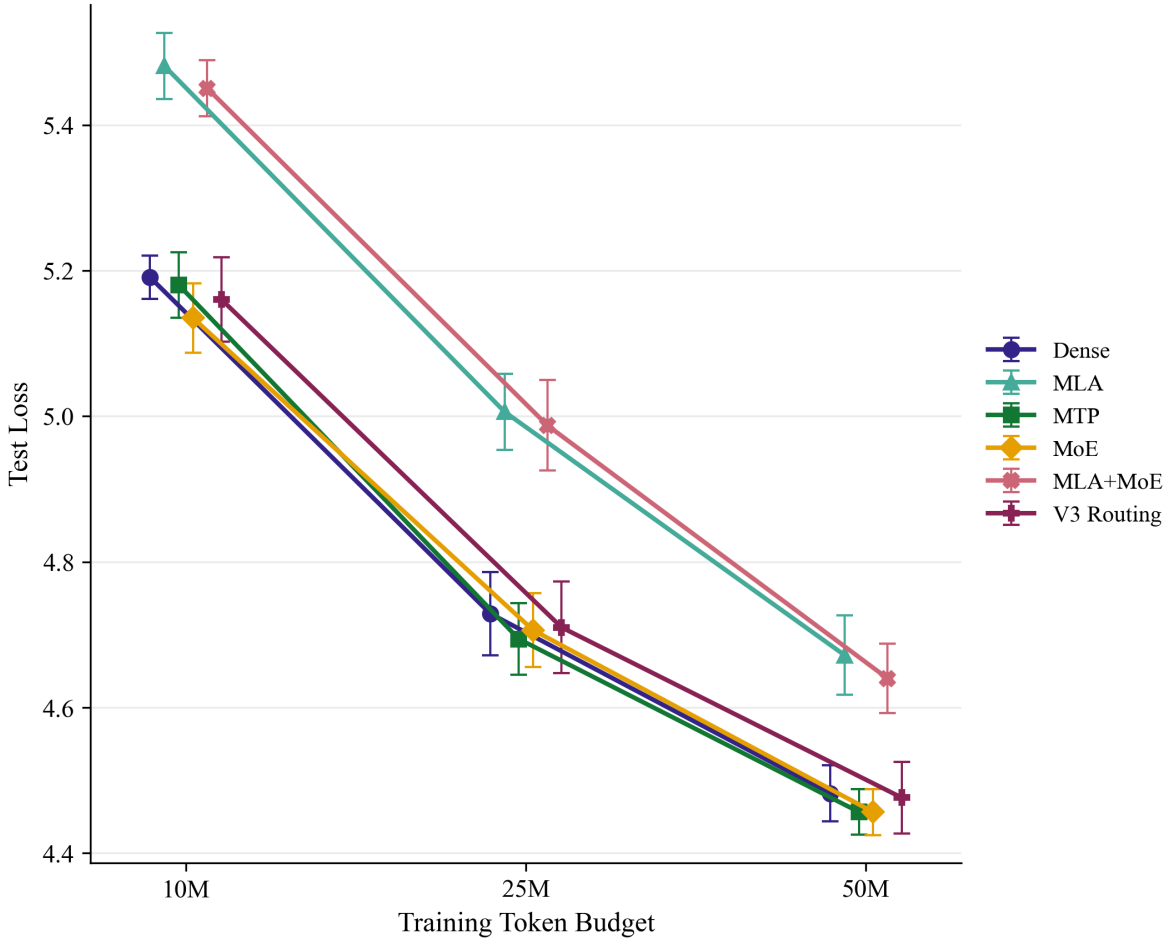


**Figure 1:** Mean test next-token cross-entropy across nominal token budgets; lower is better. Error bars are 95% Student-$t$ intervals across ten aligned seeds. Small horizontal offsets separate model markers at the same nominal budget and have no quantitative meaning. The larger-budget evaluations use nested but not identical held-out prefixes so the lines are interpreted as strong descriptive budget trends rather than perfectly identical-sample learning curves.

The ordering was more informative than any single winning mean. At 10M tokens, MoE had the lowest mean test loss (5.13507), followed closely by V3 Routing (5.16021), MTP (5.18046), and Dense (5.19078). At 50M tokens, MoE and MTP had nearly identical means (4.45648 and 4.45663) while V3 Routing and Dense remained close behind (4.47626 and 4.48224). The two MLA-based variants formed a consistently weaker group. At 50M, MLA+MoE reached 4.64007 and MLA reached 4.67192. This pattern indicates that more training data reduced loss for every model but it did not remove the architecture-specific disadvantage associated with the implemented MLA attention path.

The global tests confirmed a model effect on test loss at every budget. Both the repeated-measures ANOVA and the rank-based Friedman test were significant under within-seed permutation at 10M, 25M, and 50M tokens (Table 3). Their agreement shows that the conclusion was not limited to the

mean-based omnibus statistic.

**Table 3:** Global model-effect tests for test loss at each token budget. Both tests preserve the aligned-seed block structure. Permutation $p$-values use 20,000 within-seed model-label permutations.

| **Budget** | **RM-ANOVA** $F(5, 45)$ | **Perm.** $p$ | **Friedman** $Q(5)$ | **Perm.** $p$ |
|---|---|---|---|---|
| 10M | 113.638 | 0.000050 | 37.943 | 0.000050 |
| 25M | 87.547 | 0.000050 | 35.829 | 0.000050 |
| 50M | 56.202 | 0.000050 | 35.543 | 0.000050 |

The repeated-measures ANOVA tests mean differences among the six models; the Friedman test provides a rank-based companion. The consistent omnibus results justify interpretation of the prespecified paired contrasts rather than post hoc selection of model pairs.

The planned contrasts in Figure 2 identify where those global differences arose. Under the primary Holm family, three findings were consistent across all budgets. MLA was worse than Dense, MLA+MoE was worse than Dense, and MLA+MoE was worse than ordinary MoE. No corrected test-loss advantage was established for MoE over Dense, V3 Routing over MoE, or MTP over Dense. At 10M, the MoE–Dense contrast was suggestive—MoE was better in nine of ten seeds, with a mean difference of $-0.056$ and an exact unadjusted $p = 0.007812$—but its primary adjusted value was $p = 0.093750$. The study consequently treats this as a short-budget signal rather than a confirmed quality advantage. This implies that the broad answer to the control-model question is conditional. Dense did not have the lowest mean loss at every budget but it remained close to the best mean losses while also being the fastest and least memory-intensive model.

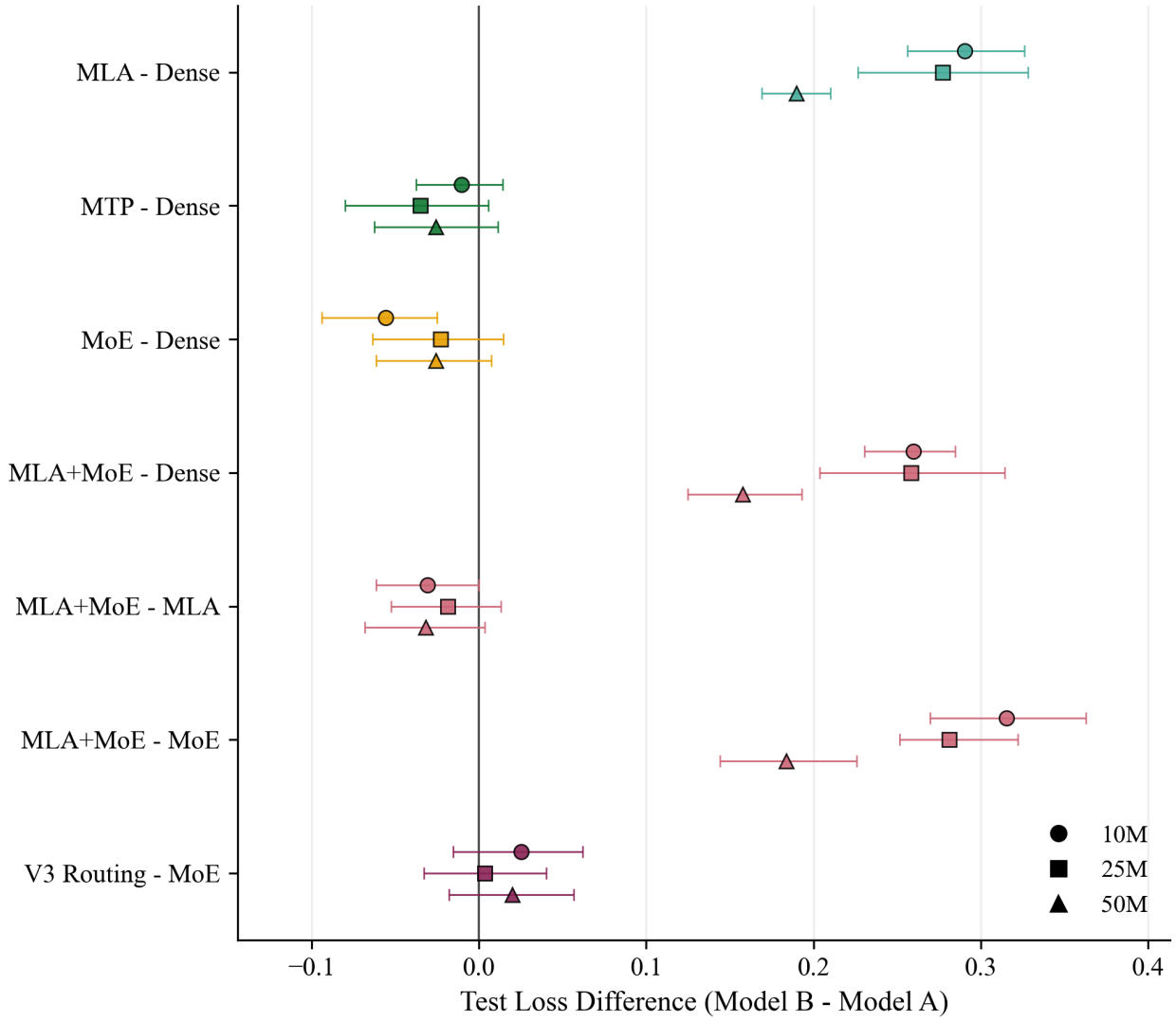


**Figure 2:** Prespecified paired test-loss contrasts across token budgets. Differences are Model B minus Model A so negative values favor Model B. Intervals are paired percentile-bootstrap 95% confidence intervals from 20,000 resamples. Budget markers are separated slightly for readability only. Statistical conclusions use the prespecified 21-comparison Holm family and are not determined solely by whether an individual bootstrap interval crosses zero.

### 4.2 MLA and Sparse-Capacity Tradeoffs

The MLA comparison answers whether the local latent-attention analogue improved training memory or throughput relative to Dense. It did not. MLA increased mean test loss by 0.290, 0.277, and 0.190 at 10M, 25M, and 50M tokens, respectively. All 30 aligned comparisons favored Dense, and each contrast remained significant under the primary Holm correction ($p = 0.041016$). MLA was also 13.9–14.2% slower and used 15.4% more peak allocated memory than Dense at every budget.

This should be interpreted against MLA's original motivation. In DeepSeek-V2, MLA was introduced primarily to compress the key–value cache during autoregressive inference, thereby reducing cache storage and memory-bandwidth pressure [3]. The present experiment measured full-sequence training, not cached autoregressive decoding. Its local MLA path projected hidden states into a latent representation and then reconstructed per-head keys and values on every forward pass; it did not use a production KV cache, FlashMLA, fused kernels, or the deployment stack that creates the original inference advantage. The MLA model also contained 137.460 million parameters compared with 120.945 million for Dense. This means extra projection work and stored parameters remained visible in training memory and wall-clock time while the intended decode-time cache benefit was not exercised.

The quality gap also narrowed from 0.290 at 10M to 0.190 at 50M. This is consistent with additional data partially reducing the optimization burden of the more elaborate attention path but the gap remained large and seed-consistent at the largest budget. Because the evaluated held-out prefix also changed with budget, the exact amount of narrowing should not be attributed to training exposure alone. The experiment cannot identify one causal source for the remaining deficit. Plausible contributors include the local latent bottleneck and projection geometry, the larger attention parameterization, the limited training regime, and the budget-dependent evaluation prefixes. The supported conclusion is narrower. The MLA analogue did not improve predictive quality, training throughput, or allocated training memory under the implemented local protocol. It does not refute MLA's production inference objective.

MoE produced a different tradeoff. The model exposed 220.146 million total parameters but activated an estimated 78.589 million per token, or 35.7% of its stored capacity. At 50M tokens, this corresponded to 0.227 tokens per total parameter but 0.636 tokens per activated parameter compared with 0.413 under both definitions for Dense. The architecture consequently achieved the intended separation between total capacity and per-token activated capacity that motivates sparse conditional computation [2, 7].

That accounting advantage did not become a confirmed quality or systems advantage. MoE's mean test loss was lower than Dense by 0.056, 0.023, and 0.026 across the three budgets but none of those differences survived the primary correction. MoE was 29.3–30.5% slower and used 51.6% more peak allocated memory. This mismatch is consistent with the distinction between activation and implementation cost. Sparse routing avoids applying every routed expert to every token but all expert parameters remain resident and trainable. In this single-GPU implementation, routing also required top-$k$ selection, indexing, repeated small expert operations, weighted aggregation, and auxiliary-loss computation without custom sparse kernels or expert parallelism. At this scale, the observed slowdown is consistent with those overheads outweighing the arithmetic saved by conditional activation.

The larger expert pool may also have been data-limited at the level of individual experts. Routed experts receive only the tokens assigned to them so total parameter count overstates how much training signal each expert receives. The shared expert and top-2 routed path can still learn useful representations, which is consistent with MoE's competitive test loss, but the retained diagnostics

do not measure expert specialization directly. This implies the answer to the sparse-routing question is mixed. MoE demonstrated sparse-capacity exposure but it did not deliver a statistically reliable quality gain, a wall-clock speedup, or a memory reduction on the local hardware.

### 4.3 Interaction Between MLA and MoE

The combined MLA+MoE model completed all 30 planned runs so the two mechanisms were technically composable but they did not compound their intended benefits. Relative to ordinary MoE, MLA+MoE increased test loss by 0.315, 0.281, and 0.184 across the three budgets; every seed favored MoE, and all three primary Holm values were $p = 0.041016$. Relative to Dense, the combined model was also worse at every budget. By contrast, MLA+MoE differed only slightly from MLA itself ($-0.031$, $-0.019$, and $-0.032$), and none of those differences were significant under the primary family.

The comparison is consistent with the combined model inheriting the dominant behavior of the MLA path. Adding sparse experts increased total and activated capacity but it did not recover the quality gap introduced relative to dense attention. At the same time, MLA+MoE retained the systems costs of both mechanisms. It was 8.9–10.3% slower than MoE and used 7.1% more peak memory, with 236.662 million total and 95.104 million activated parameters. In practical terms, the sparse feed-forward path did not offset the local attention penalty and the latent-attention path added cost to an already expensive sparse implementation.

This is evidence against a compounding benefit in the tested configuration, not proof of an inherently antagonistic architectural interaction. The models were not constructed as a fully crossed, matched-capacity factorial design so parameter count and mechanism cannot be separated perfectly. The defensible conclusion is that the selected V2-style components remained trainable together but provided no combined quality or efficiency benefit under these settings.

### 4.4 V3-Style Routing Behavior

V3 Routing held the MoE expert geometry fixed and replaced the differentiable load-balancing loss with a non-gradient expert-selection bias. Its test-loss differences relative to ordinary MoE were 0.025, 0.004, and 0.020 from 10M through 50M tokens. The adjusted $p$-value was 1.000000 at all three budgets. This implies the routing change did not improve next-token quality.

It produced a small systems effect. V3 Routing was 0.8%, 0.5%, and 1.9% faster than MoE across the three budgets. The first two differences survived the corresponding all-budget throughput correction while the 50M difference did not ($p = 0.095703$). Peak allocated memory was only 0.2% higher. Removing the auxiliary-loss term plausibly reduced a small amount of forward and backward work but the two models retained the same experts, router, and dispatch path; a large speed or memory change was not expected.

Figure 3 shows how the routing distributions differed. Routing entropy was higher in the longer-budget runs for all three sparse models, indicating that their final router probabilities were less concentrated. At 50M, mean entropy was 1.95095 for MoE and 1.91579 for V3 Routing. The more important balance measure was the variance of actual top-2 selections. V3 Routing's expert-load variance decreased from 0.000732 at 10M to 0.000658 at 50M, a paired reduction of 0.000074 (unadjusted exact $p = 0.041016$) but its final variance remained descriptively above ordinary MoE's 0.000404. Because of this, longer-budget V3 runs ended with more even allocations than 10M runs while still producing less balanced terminal selections than the auxiliary-loss router.

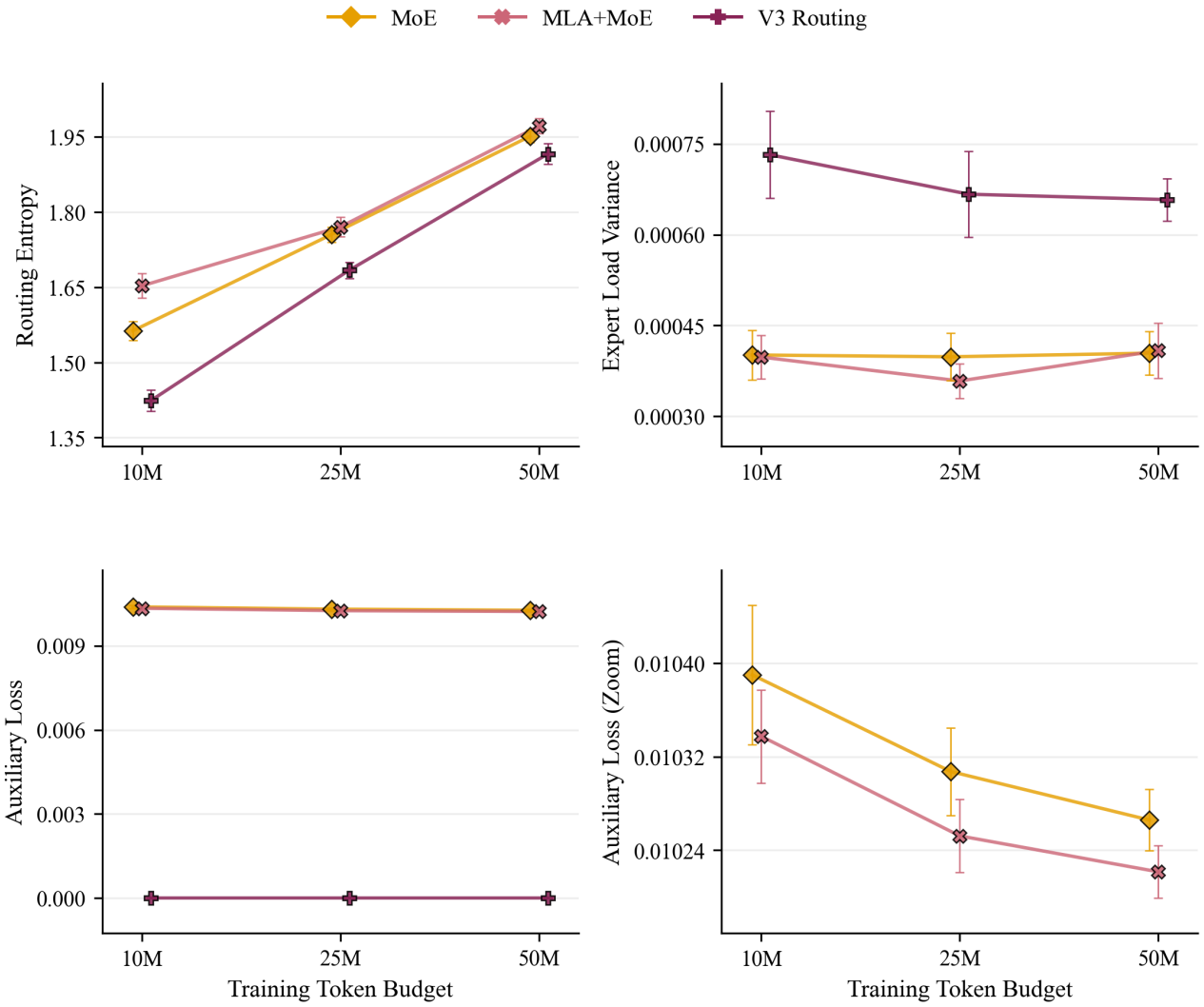


**Figure 3:** Terminal routing diagnostics for the three sparse models. Values come from the latest forward pass of the final evaluated test batch, averaged across 12 MoE layers and then across ten seeds; error bars are 95% Student-$t$ intervals. Small model offsets are visual only. The expert-load-variance panel is zoomed to distinguish low-magnitude values (approximately 0.00025–0.00085); the underlying values are unchanged. V3 auxiliary loss is zero by design.

This outcome is consistent with the difference between the two controllers. The auxiliary-loss model receives direct gradient pressure linking routing probabilities and expert load, whereas V3 Routing adjusts only a small selection bias after each training forward. DeepSeek-V3 introduced auxiliary-loss-free balancing to avoid the possibility that a balancing objective interferes with the language-model objective at large scale [4]. In the local experiment, removing that objective did not improve quality, and the selection-bias controller did not match the terminal balance of the ordinary MoE router. This does not establish routing collapse or failed specialization. Lower entropy can reflect focused routing and the recorded values are final-batch snapshots rather than training trajectories. The supported answer is that the implemented V3-style controller slightly reduced runtime overhead, did not improve predictive quality, and ended with higher terminal expert-load variance than the auxiliary-loss MoE baseline.

### 4.5 Multi-Token Prediction

MTP tested whether two auxiliary future-token heads improved optimization or sample efficiency. Its mean test-loss differences relative to Dense were $-0.010$, $-0.035$, and $-0.026$ at 10M, 25M, and 50M tokens. The directions favored MTP but the bootstrap intervals crossed zero and all primary Holm values were 1.000000. MTP won five, seven, and six of the ten seed pairs, respectively. This suggests the experiment did not establish a reliable next-token quality advantage.

The auxiliary loss decreased as the training budget increased. As shown in Figure 4, final-step MTP loss decreased from 5.50872 at 10M to 5.14024 at 50M; the paired 50M-minus-10M change was $-0.368$ with unadjusted exact $p = 0.019531$. This means that the future-token heads improved on their own objective as the budget increased. It does not show faster convergence of the shared language model because the diagnostic is the final optimizer-step value, not a complete trajectory, and its scale is not directly comparable with test next-token loss.

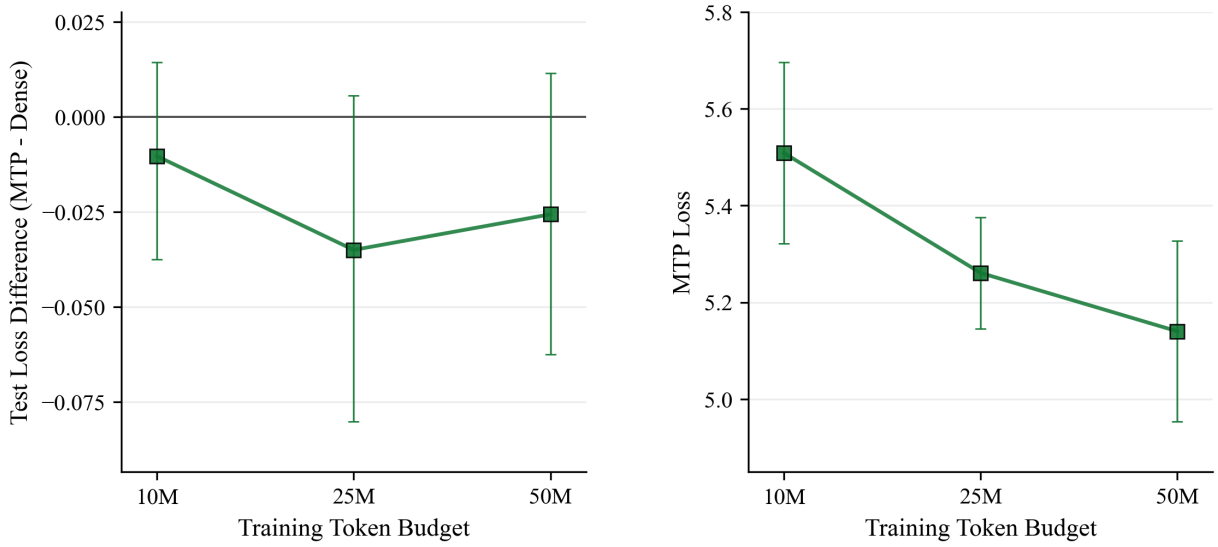


**Figure 4:** MTP behavior across token budgets. Left: paired MTP-minus-Dense test-loss differences with 20,000-resample percentile-bootstrap 95% intervals; negative favors MTP. Right: final optimizer-step MTP auxiliary loss with 95% Student-$t$ intervals across ten seeds.

The missing transfer to next-token quality is informative. Multi-token prediction is motivated as a denser training signal that can improve sample efficiency, with prior work reporting larger benefits as model scale grows and especially on generative coding tasks [8]. Here, the 136.305-million-parameter model was trained for at most 50M tokens and evaluated only by next-token cross-entropy. The two full-vocabulary auxiliary heads added 15.360 million parameters relative to Dense, increased peak memory by 12.9%, and reduced throughput by 12.7–13.1%. A plausible interpretation is that the auxiliary heads learned the harder future-token objectives but the resulting gradient signal was not strong or mature enough to improve the shared representation reliably within this scale and budget. The answer to the MTP research question is consequently limited but clear. The auxiliary objective was optimized, yet no statistically reliable improvement in ordinary next-token quality or sample efficiency was demonstrated, and the local implementation imposed measurable training cost.

### 4.6 Cross-Mechanism Quality, Systems Cost, and Exposure Synthesis

The 50M endpoint summarizes the practical tradeoff among the six models. Figure 5 shows that the lowest mean losses belonged to MTP and MoE but they achieved those means with very different systems profiles. Dense remained the fastest model at 4,336.0 tokens/s and used the least peak memory at 2.83 GiB while its mean test loss was only 0.026 above MoE and MTP. MLA was both slower and less accurate than Dense. MLA+MoE occupied the least favorable region, combining relatively high loss with the lowest throughput. V3 Routing modestly improved MoE throughput but did not improve its quality.

**Table 4:** Quality, systems cost, and parameter exposure at the 50M-token budget. Values are means across ten aligned seeds; test-loss intervals are 95% Student-$t$ intervals. Throughput is training target tokens per measured end-to-end run second, and peak memory is maximum CUDA memory allocated by PyTorch.

| Model | Test loss [95% CI] | Test PPL | Tokens/s | Peak GiB | Tokens/total | Tokens/activated |
|---|---|---|---|---|---|---|
| Dense | 4.48224 [4.44340, 4.52109] | 88.55136 | 4336.0 | 2.83 | 0.413 | 0.413 |
| MLA | 4.67192 [4.61745, 4.72639] | 107.18503 | 3733.8 | 3.27 | 0.364 | 0.364 |
| MTP | 4.45663 [4.42555, 4.48771] | 86.26896 | 3768.9 | 3.20 | 0.367 | 0.367 |
| MoE | 4.45648 [4.42500, 4.48796] | 86.25870 | 3012.5 | 4.29 | 0.227 | 0.636 |
| MLA+MoE | 4.64007 [4.59278, 4.68737] | 103.75763 | 2743.5 | 4.60 | 0.211 | 0.526 |
| V3 Routing | 4.47626 [4.42683, 4.52570] | 88.09388 | 3070.3 | 4.30 | 0.227 | 0.636 |

All parameters were trainable, so tokens per trainable parameter equal tokens per total parameter. Exposure ratios use the actual 50M-run token count of 50,000,896. Activated parameters are an analytical per-token architecture estimate, not measured FLOPs.

Parameter exposure explains why total model size alone is insufficient for this comparison. Figure 6 separates tokens per total parameter from tokens per activated parameter. The two measures coincide for Dense, MLA, and MTP because every parameter participates in every token path. They diverge sharply for MoE and V3 Routing. Each stored 220.146 million parameters but only an estimated 78.589 million were activated per token. MLA+MoE showed the same pattern at a higher activated count.

This distinction changes the interpretation of sparse capacity. At 50M, MoE had fewer tokens per total parameter than Dense (0.227 versus 0.413), making it more data-limited relative to stored capacity. It simultaneously had more tokens per activated parameter (0.636 versus 0.413) because each token used only part of the expert pool. Due to this, sparse models can appear heavily overparameterized under total capacity while receiving greater exposure per active path. Neither ratio alone determines quality. Figure 7 shows that models with similar activated exposure still differed in loss and models with different exposure could achieve similar loss.

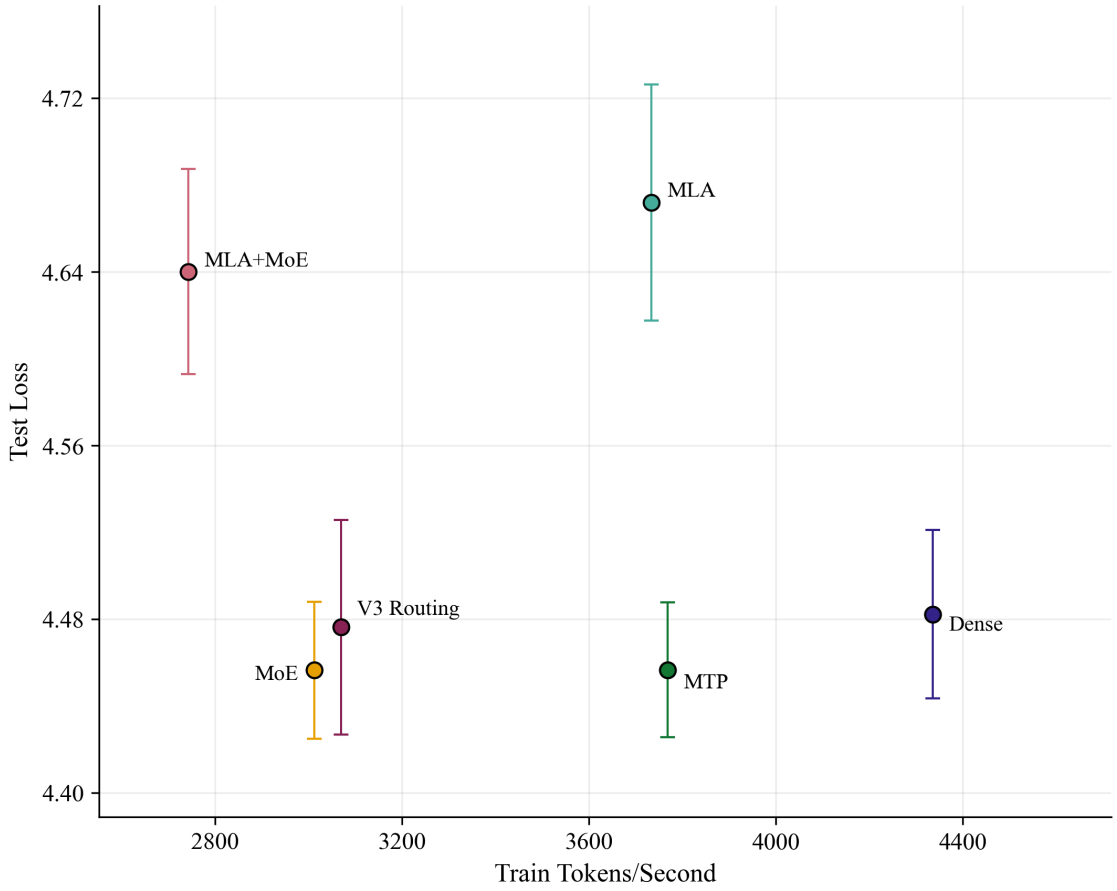


**Figure 5:** Quality–throughput tradeoff at the 50M-token budget. The horizontal axis is mean training-target tokens per measured end-to-end run second; the vertical axis is mean test loss, with 95% Student-$t$ intervals across ten seeds. Higher throughput and lower loss are favorable but the plot does not define a single scalar ranking. Horizontal uncertainty is not shown.

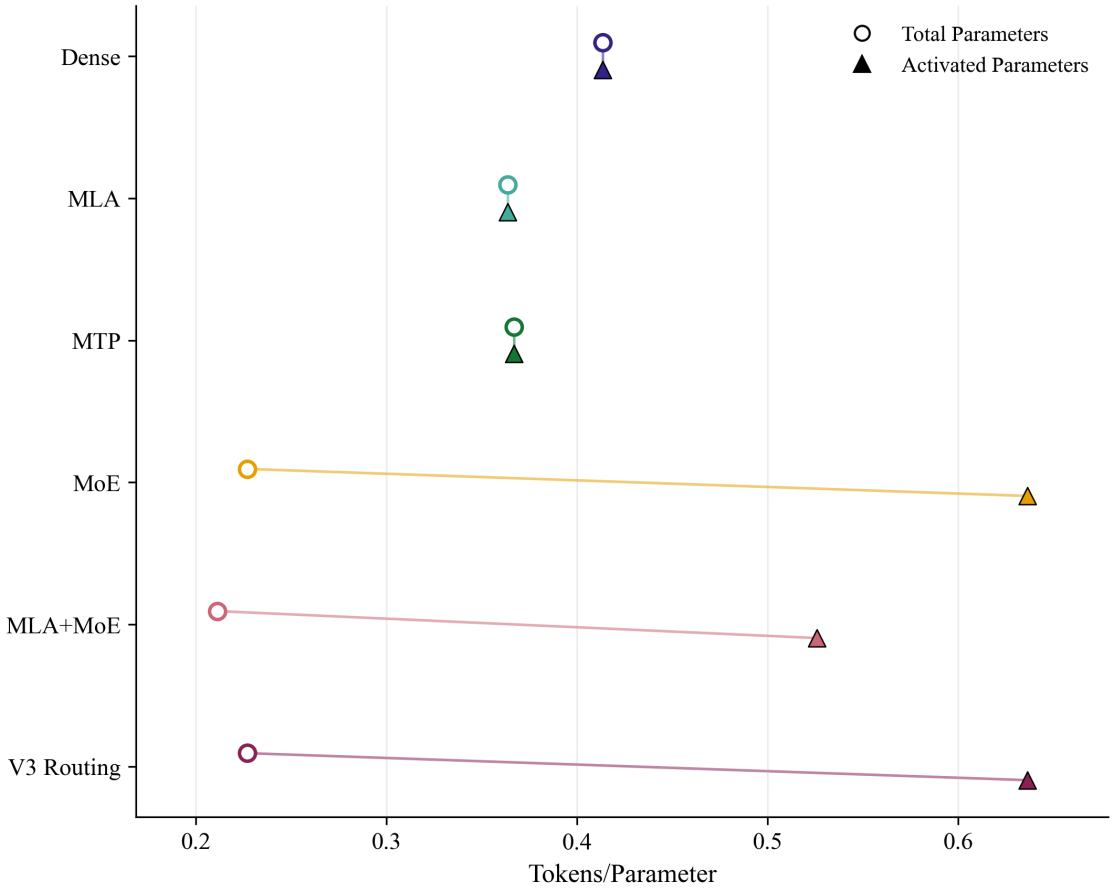


**Figure 6:** Total-capacity and activated-compute exposure at 50M tokens. Values are deterministic for a given architecture and processed-token count so no uncertainty intervals are shown. Marker offsets distinguish the two exposure definitions within each model and do not represent different budgets. Total and trainable exposure are identical because all parameters were trainable.

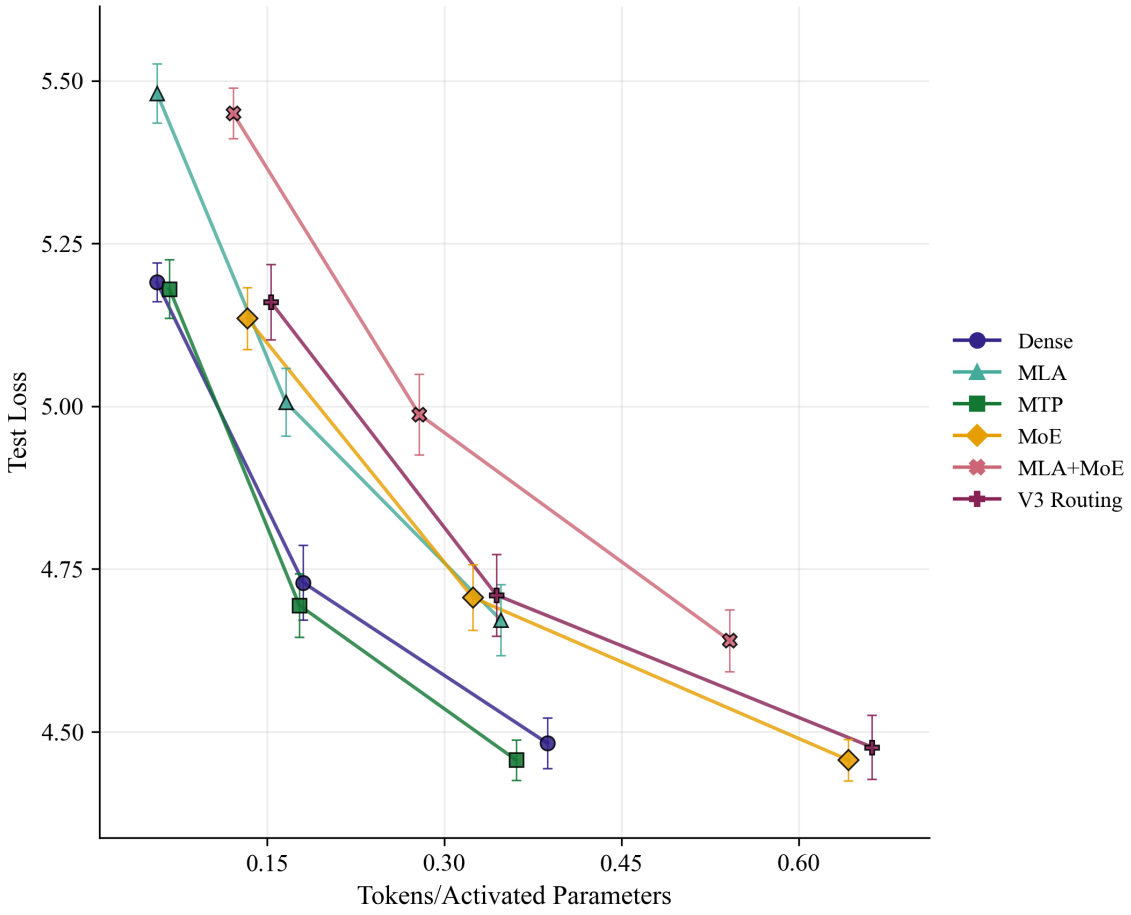


**Figure 7:** Mean test loss versus tokens per activated parameter across models and budgets. Vertical bars are 95% Student-$t$ intervals across ten seeds; small horizontal offsets are display-only.

Taken together, the results answer the central research question in mechanism-specific terms. Dense provided the strongest overall balance of quality, speed, memory use, and implementation simplicity. MoE successfully separated stored from activated capacity and remained quality-competitive but naive local sparse execution was slower and more memory-intensive. The MLA analogue did not realize its intended efficiency benefit in the measured training setting and produced a persistent quality penalty. Combining MLA with MoE compounded systems cost without improving quality. V3-style bias routing slightly reduced MoE runtime overhead but did not improve quality and ended with higher terminal expert-load variance than ordinary MoE. MTP reduced its auxiliary loss without delivering a reliable next-token benefit within the tested scale.

The strongest general pattern was not a universal architectural winner but the value of additional training tokens. Every model improved in every aligned seed, with the loss decreasing monotonically across the three budgets, but the study did not include a same-architecture capacity sweep dense enough to identify an interpolation threshold or support classical double-descent claims. As a result, the findings should not be viewed as a general scaling law or reproduction of DeepSeek efficiency and instead should be read as controlled local evidence about mechanism tradeoffs in a data-limited, overparameterized regime.

## 5. Limitations and Threats to Validity

The study was designed to isolate several DeepSeek-inspired mechanisms under a controlled local protocol, not to reproduce the production systems from which those mechanisms were drawn. The balanced design, aligned seeds, and artifacts strengthen the internal comparisons but they do not remove the limits imposed by model scale, implementation choices, measurement design, and hardware. Due to this, conclusions should be read as evidence about the six implemented systems within this regime.

### 5.1 Scale and Implementation Fidelity

The models were trained at approximately 121–237 million total parameters, with a 256-token context and at most 50 million training tokens. These settings create a deliberately data-limited and overparameterized local regime but they are far from the scale, context length, data diversity, and optimization infrastructure used by production language models. A mechanism that is ineffective at this scale may become useful when communication, memory bandwidth, cache size, or expert specialization becomes the dominant constraint. Conversely, a local gain need not persist when the same mechanism is distributed across many devices.

The implemented MLA variant was a local latent-projection analogue rather than the complete DeepSeek-V2 attention stack. It did not evaluate autoregressive key–value cache size, decode latency, FlashMLA, fused kernels, or production cache management. This means negative training-memory and throughput results apply to the implemented full-sequence training path, not to MLA's intended inference-time cache advantage. The sparse models likewise omitted expert parallelism, distributed all-to-all communication, custom sparse kernels, capacity enforcement, and token dropping. Their measured overhead reflects a single-GPU implementation in which all experts remained resident and routed computation was executed through general PyTorch operations. It should not be interpreted as a benchmark of a production MoE systems stack.

The MTP implementation used two separate auxiliary output heads. It did not reproduce shared-head

variants, generation-time token verification, speculative decoding, instruction tuning, reinforcement learning, or reasoning evaluation. This means the experiment can determine whether the auxiliary objective changed next-token learning within the tested budget but it cannot determine whether MTP would improve decoding efficiency or downstream reasoning behavior.

### 5.2 Measurement and Evaluation Design

Training examples were random windows sampled with replacement from one tokenized corpus. Windows could overlap or repeat so processed tokens were dependent language-model targets rather than independent observations. The statistical analysis correctly treated the ten aligned seeds as the experimental units but the effective diversity of the training signal was still constrained by a single corpus, tokenizer, and document sample.

As described in Section 3.2, validation and test losses were computed from heavily overlapping stride-one windows. The evaluated prefixes were shared within a budget but increased from 10M to 50M. This preserves the validity of paired model contrasts at a fixed budget while making cross-budget changes slightly less clean than learning curves evaluated on an identical held-out sample. The consistent direction across all seeds supports the reported budget effect but exact cross-budget magnitudes partly reflect the larger evaluated prefix.

The systems measurements also have narrow meanings. Throughput counts training target tokens divided by the complete measured run time, which includes logging and evaluation but excludes model construction and data-loader setup. It is most directly comparable among models within the same budget and software environment; it is not pure optimizer-step throughput. Peak memory records maximum CUDA tensor memory allocated by PyTorch, not reserved device memory, host memory, or the memory consumption of other processes. Activated parameter count is an analytical architecture-level estimate rather than measured floating-point work, communication, or kernel efficiency. These distinctions are particularly important for MoE, where reduced parameter activation did not translate directly into lower wall-clock time or allocated memory.

The retained mechanism diagnostics are terminal summaries rather than training trajectories. Routing entropy and expert-load variance were measured from the final evaluated test batch and averaged across layers while MTP loss was taken from the final optimizer step. They can describe the state reached by each implementation but they cannot establish when routing stabilized, whether experts developed persistent specialization, or whether the MTP objective changed convergence speed. Trajectory-level logging or checkpointed evaluations would be required for those claims.

Finally, ten seeds provide a substantially stronger basis than a single-run comparison but they still limit inferential resolution. The exact two-sided sign-flip test has a minimum nonzero value of 0.001953, and confidence intervals can remain wide for small effects. Holm correction appropriately controls the complete primary family but also reduces power. This means a nonsignificant contrast should be interpreted as a failure to establish an advantage rather than as proof that the two models are equivalent.

### 5.3 RoPE Implementation Limitation

The retained rotary-position implementation contains a frequency-layout mismatch. Its cosine and sine frequencies are formed by concatenating the frequency vector while the rotation operator pairs adjacent even and odd dimensions. Adjacent rotated features can therefore receive different frequencies

rather than the repeated frequency pair required by the intended rotary construction.

The same RoPE utility was used across all six model families so the issue was not selectively introduced into one comparison. Shared exposure, however, does not guarantee an equal effect because the dense-attention and MLA paths use positional components differently. The reported contrasts remain valid comparisons among the implemented systems but absolute quality and attention-specific conclusions should not be treated as estimates under a verified RoPE implementation. A corrected rerun would be required to determine whether any model ordering changes.

### 5.4 Generalization and Untested Claims

All final runs used full 32-bit precision on one NVIDIA RTX 4050 GPU under one Windows, CUDA, and PyTorch environment. The throughput and memory results are consequently hardware-specific and implementation-specific. The study also used one FineWeb-Edu-derived corpus sample, one 10,000-token BPE tokenizer, one context length, one optimizer configuration, and three short-to-medium token budgets. Generalization across corpora, tokenizers, sequence lengths, precision formats, accelerators, or larger training budgets remains untested.

The experiment contains two broad total-parameter scales but capacity is confounded with mechanism. This means the larger models are the sparse variants rather than larger dense versions of the same architecture. This means the matrix cannot identify an interpolation threshold, estimate a capacity scaling curve, or establish classical double descent.

The paired design supports controlled empirical comparisons but it does not by itself prove the causal mechanism behind every observed difference. For example, MLA changes both attention structure and parameter count while MoE changes feed-forward structure, total capacity, routing, and execution pattern. The explanations in Section 4 are consistent with the implementation and measurements but additional ablations would be needed to isolate projection dimensions, expert counts, top-$k$, auxiliary-loss strength, bias-update rate, MTP weight, or parameter-matched controls.

Seeded configurations and retained artifacts support protocol-level reproducibility but do not guarantee bitwise identity across machines. Full checkpoint-based interruption and resumption was also not implemented. More broadly, the study does not evaluate instruction following, downstream tasks, calibration, generation quality, inference latency, speculative decoding, or reasoning. Its conclusions are limited to next-token prediction, measured local training cost, parameter exposure, and terminal mechanism diagnostics under the canonical training protocol.

## 6. Conclusion

### 6.1 Main Findings

This study evaluated six DeepSeek-inspired decoder variants across three token budgets and ten aligned seeds, producing a balanced set of 180 completed runs. The central result is not that one efficiency mechanism dominated every metric. Rather, the mechanisms redistributed stored capacity, activated capacity, optimization signal, memory demand, and execution overhead in different ways. Their value depended on the bottleneck being measured and on whether the local implementation exercised the systems advantage for which the mechanism was originally designed.

The dense model provided the strongest overall control. It remained close to the best mean test loss at every budget while achieving the highest throughput, the lowest peak allocated memory, and the

simplest execution path. Additional training tokens produced the most robust improvement in the study. All six model families improved from 10M to 50M tokens for every aligned seed. Within this data-limited regime, increasing exposure to the training corpus was more consistently beneficial than adding architectural complexity.

The MLA analogue did not improve any measured training-efficiency outcome. Relative to Dense, it produced higher test loss, lower throughput, and greater allocated memory at all three budgets. The gap narrowed as the token budget increased but it remained substantial and seed-consistent at 50M tokens. This finding applies to the implemented full-sequence training path. It does not test or reject MLA's intended production benefit of reducing the autoregressive key–value cache because cached decoding, FlashMLA, and fused inference kernels were outside the experiment.

Sparse MoE routing achieved its main architectural accounting objective, meaning it separated total stored capacity from the smaller subset of parameters activated for each token. Under activated-parameter exposure, MoE used a larger effective token-to-capacity ratio than the dense control while retaining substantially more total capacity. This did not translate into a confirmed test-loss advantage under the primary corrected analysis, nor did it reduce local wall-clock time or memory use. All experts remained resident on one GPU, and the general PyTorch routing path introduced top-$k$ selection, indexing, dispatch, and aggregation overhead. This implies that the result is mixed rather than negative. Sparse capacity was realized but the local systems stack did not convert that property into faster or cheaper training.

Combining MLA with MoE demonstrated technical composability but not compounded benefit. The combined model completed every planned run, yet its quality remained close to MLA and clearly worse than ordinary MoE, while its throughput and memory profile were the least favorable among the tested variants. V3-style expert-bias routing also failed to establish a quality advantage over auxiliary-loss routing. It modestly reduced MoE runtime overhead but its terminal expert-load variance was descriptively higher and its test loss was not reliably better. These terminal diagnostics do not prove routing collapse or failed specialization; they show only that the implemented bias controller did not improve final predictive quality or measured load balance within the tested regime.

MTP reduced its final auxiliary future-token loss as training increased but that improvement did not transfer reliably to ordinary next-token prediction. Its mean test loss was slightly lower than Dense at each budget, yet none of the paired differences survived the prespecified Holm correction. The auxiliary heads also increased parameter count, memory use, and training time. The experiment therefore does not establish improved sample efficiency at this scale, and it provides no evidence about speculative decoding or reasoning—which were not evaluated.

The parameter-exposure analysis clarifies why these results cannot be reduced to a single model-size ranking. Tokens per total parameter describe how strongly stored capacity is exposed to data, whereas tokens per activated parameter approximate exposure of the path used for each token. These quantities coincide for the dense, MLA, and MTP models but diverge sharply for sparse models. Neither exposure measure alone predicted quality. Architecture, routing, optimization, and expert-specific data allocation remained important.

Taken together, the evidence answers the central research question in mechanism-specific terms. Under the local protocol, Dense offered the best balance of predictive quality, throughput, memory, and simplicity. MoE provided meaningful sparse-capacity accounting but required systems support that was absent from the single-GPU implementation. MLA's intended inference advantage was not exercised and its local training path imposed a clear penalty. V3-style routing and MTP remained plausible mechanisms whose expected benefits were not established within the available scale and token budgets.

The broader lesson is that architectural motivation should not be treated as an empirical benefit because quality, systems cost, parameter exposure, and mechanism behavior must be measured separately.

### 6.2 Implications and Future Work

The practical implication is that mechanism selection should follow the actual computational constraint. For local full-sequence training, the dense model was difficult to improve upon because its regular tensor operations were efficient and its quality was already competitive. Sparse routing becomes more attractive when stored capacity is valuable and the execution environment can exploit conditional computation through expert parallelism, optimized dispatch, and fused kernels. MLA should be judged primarily with long-context autoregressive benchmarks that measure key–value cache size, decode latency, and memory bandwidth. MTP should be evaluated over longer training horizons and with generation-time objectives before conclusions are drawn about decoding efficiency or reasoning.

The first methodological priority is to correct the retained RoPE frequency-layout mismatch and rerun the complete balanced experiment matrix. Because every model used the same utility but the dense-attention and MLA paths may respond differently, only a corrected rerun can determine whether the observed ordering is unchanged.

A second priority is to separate mechanism effects from capacity and data exposure more cleanly. A same-architecture capacity sweep would be required to study interpolation behavior or double descent. Larger token budgets would test whether the MLA quality gap continues to narrow, whether routed experts receive enough assignments to specialize, and whether MTP's auxiliary signal eventually improves next-token prediction. Repeating the study across corpora, tokenizer sizes, context lengths, and precision formats would also show which findings are specific to the current FineWeb-Edu sample and RTX 4050 environment.

A third direction is systems-oriented evaluation. MLA should be benchmarked with an actual autoregressive cache and optimized latent-attention kernels. MoE should be tested with expert parallelism, efficient all-to-all communication, capacity management, token dropping where appropriate, and specialized sparse kernels. Those experiments would distinguish architectural conditional-computation benefits from the overhead of the current eager single-GPU implementation. Inference latency, memory consumption, and energy use would provide a more complete efficiency assessment than training throughput alone.

Finally, richer diagnostics would strengthen the mechanism-level explanations. Checkpointed routing trajectories, per-expert token counts, specialization probes, and gradient measurements could show how expert utilization develops rather than only recording the final batch. MTP learning curves and auxiliary-weight ablations could determine whether the absent next-token gain reflects insufficient training, objective weighting, or model scale. Reasoning-oriented supervised fine-tuning or reinforcement learning should remain a later extension, undertaken only after the base architectural results have been rerun with corrected positional encoding and validated under a broader experimental regime.

## A. Full Descriptive Results

This appendix preserves the exact descriptive record needed to evaluate the main quality, systems, and parameter-exposure findings. It includes all 18 model–budget cells.

**Table A.1:** Full validation and test quality descriptives. Loss entries are means with 95% Student-$t$ intervals across ten aligned seeds; perplexity entries are means. Lower values are better.

| Budget | Model | Validation loss [95% CI] | Test loss [95% CI] | Validation PPL | Test PPL |
|---|---|---|---|---|---|
| 10M | Dense | 4.89284 [4.83363, 4.95205] | 5.19078 [5.16125, 5.22031] | 133.74091 | 179.74811 |
| 10M | MLA | 5.13541 [5.08378, 5.18704] | 5.48108 [5.43543, 5.52672] | 170.33008 | 240.54322 |
| 10M | MTP | 4.88430 [4.82817, 4.94044] | 5.18046 [5.13554, 5.22537] | 132.55872 | 178.07568 |
| 10M | MoE | 4.85016 [4.79639, 4.90394] | 5.13507 [5.08752, 5.18261] | 128.07853 | 170.20553 |
| 10M | MLA+MoE | 5.10973 [5.05474, 5.16472] | 5.45040 [5.41185, 5.48895] | 166.06778 | 233.14991 |
| 10M | V3 Routing | 4.87967 [4.82417, 4.93517] | 5.16021 [5.10224, 5.21817] | 131.94218 | 174.71772 |
| 25M | Dense | 4.51793 [4.48309, 4.55277] | 4.72909 [4.67191, 4.78627] | 91.74388 | 113.52134 |
| 25M | MLA | 4.79847 [4.76297, 4.83396] | 5.00626 [4.95416, 5.05836] | 121.45949 | 149.70253 |
| 25M | MTP | 4.49843 [4.47176, 4.52509] | 4.69410 [4.64495, 4.74325] | 89.93079 | 109.53647 |
| 25M | MoE | 4.48119 [4.44997, 4.51241] | 4.70636 [4.65588, 4.75684] | 88.41611 | 110.89667 |
| 25M | MLA+MoE | 4.76894 [4.73767, 4.80021] | 4.98757 [4.92562, 5.04952] | 117.89531 | 147.06691 |
| 25M | V3 Routing | 4.49686 [4.46584, 4.52789] | 4.71004 [4.64721, 4.77287] | 89.80987 | 111.44679 |
| 50M | Dense | 4.31528 [4.28066, 4.34990] | 4.48224 [4.44340, 4.52109] | 74.91303 | 88.55136 |
| 50M | MLA | 4.52036 [4.46981, 4.57091] | 4.67192 [4.61745, 4.72639] | 92.07667 | 107.18503 |
| 50M | MTP | 4.27516 [4.24345, 4.30687] | 4.45663 [4.42555, 4.48771] | 71.95394 | 86.26896 |
| 50M | MoE | 4.27534 [4.24141, 4.30928] | 4.45648 [4.42500, 4.48796] | 71.97710 | 86.25870 |
| 50M | MLA+MoE | 4.50037 [4.46415, 4.53660] | 4.64007 [4.59278, 4.68737] | 90.15448 | 103.75763 |
| 50M | V3 Routing | 4.28504 [4.25214, 4.31795] | 4.47626 [4.42683, 4.52570] | 72.67545 | 88.09388 |

*Note.* Perplexity is exp(loss) and is therefore a monotonic restatement of cross-entropy rather than an independent inferential endpoint. Evaluation used deterministic, overlapping stride-one windows; the evaluated prefix increased with the token budget, as described in Section 3.

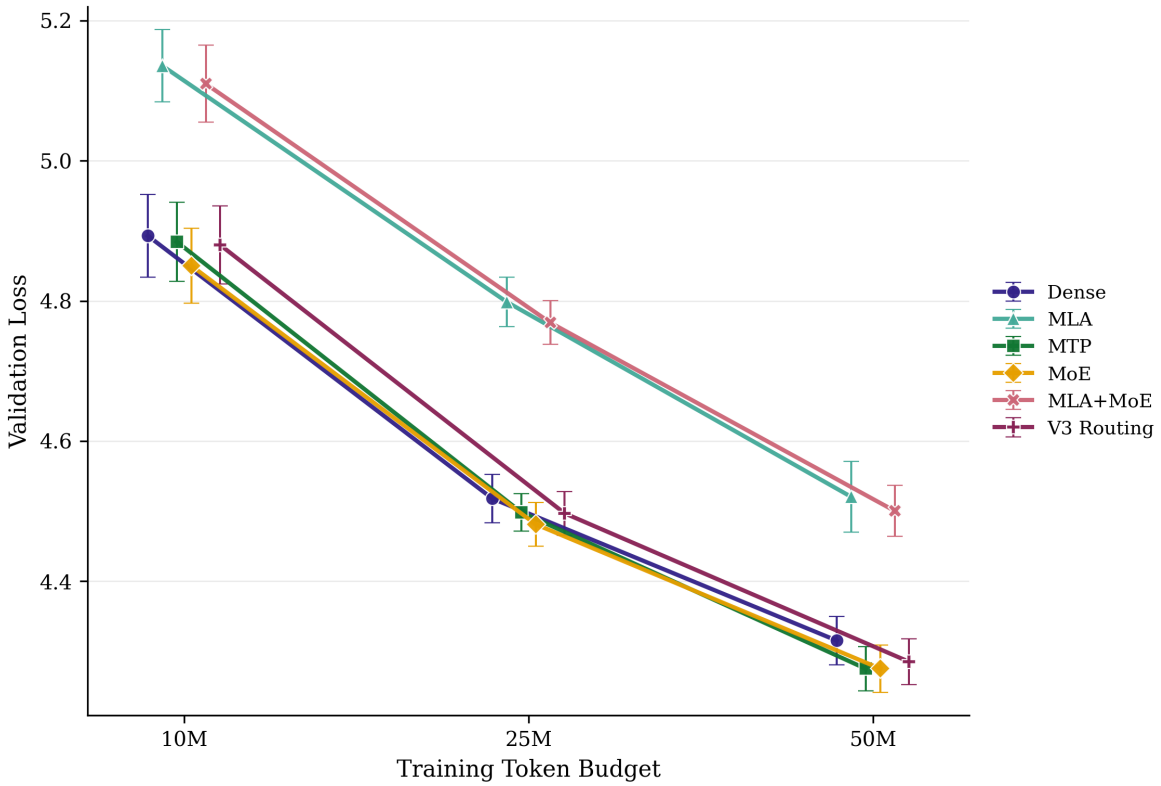


**Figure A.1:** Validation loss across token budgets. Points are means across ten aligned seeds and bars are 95% Student-$t$ intervals; lower is better. Small horizontal offsets separate models at the same nominal budget and have no statistical meaning. Evaluation uses overlapping stride-one windows with budget-dependent nested prefixes.

**Table A.2:** Full systems and parameter-exposure descriptives. Throughput entries are means with 95% Student-$t$ intervals across ten aligned seeds. Peak-memory entries are means; their intervals are omitted because all bounds round to the displayed two-decimal GiB values. Parameter counts and exposure values are deterministic at a fixed architecture and executed token budget.

| Budget | Model | Tokens/s [95% CI] | Peak GiB | Total M | Activated M | Tokens/total | Tokens/activated |
|---|---|---|---|---|---|---|---|
| 10M | Dense | 4278.7 [4250.7, 4306.8] | 2.83 | 120.945 | 120.945 | 0.083 | 0.083 |
| 10M | MLA | 3684.4 [3666.1, 3702.6] | 3.27 | 137.460 | 137.460 | 0.073 | 0.073 |
| 10M | MTP | 3735.9 [3717.5, 3754.2] | 3.20 | 136.305 | 136.305 | 0.073 | 0.073 |
| 10M | MoE | 3024.0 [3011.0, 3037.0] | 4.29 | 220.146 | 78.589 | 0.045 | 0.127 |
| 10M | MLA+MoE | 2719.4 [2709.4, 2729.4] | 4.60 | 236.662 | 95.104 | 0.042 | 0.105 |
| 10M | V3 Routing | 3049.5 [3036.0, 3063.0] | 4.30 | 220.146 | 78.589 | 0.045 | 0.127 |
| 25M | Dense | 4342.2 [4334.5, 4349.9] | 2.83 | 120.945 | 120.945 | 0.207 | 0.207 |
| 25M | MLA | 3725.5 [3717.4, 3733.6] | 3.27 | 137.460 | 137.460 | 0.182 | 0.182 |
| 25M | MTP | 3772.8 [3764.0, 3781.7] | 3.20 | 136.305 | 136.305 | 0.183 | 0.183 |
| 25M | MoE | 3052.1 [3043.9, 3060.3] | 4.29 | 220.146 | 78.589 | 0.114 | 0.318 |
| 25M | MLA+MoE | 2739.0 [2733.2, 2744.7] | 4.60 | 236.662 | 95.104 | 0.106 | 0.263 |
| 25M | V3 Routing | 3067.5 [3063.0, 3072.0] | 4.30 | 220.146 | 78.589 | 0.114 | 0.318 |
| 50M | Dense | 4336.0 [4305.4, 4366.7] | 2.83 | 120.945 | 120.945 | 0.413 | 0.413 |
| 50M | MLA | 3733.8 [3724.9, 3742.7] | 3.27 | 137.460 | 137.460 | 0.364 | 0.364 |
| 50M | MTP | 3768.9 [3742.6, 3795.3] | 3.20 | 136.305 | 136.305 | 0.367 | 0.367 |
| 50M | MoE | 3012.5 [2905.6, 3119.3] | 4.29 | 220.146 | 78.589 | 0.227 | 0.636 |
| 50M | MLA+MoE | 2743.5 [2739.6, 2747.5] | 4.60 | 236.662 | 95.104 | 0.211 | 0.526 |
| 50M | V3 Routing | 3070.3 [3054.2, 3086.4] | 4.30 | 220.146 | 78.589 | 0.227 | 0.636 |

*Note.* Throughput is training target tokens divided by measured end-to-end run time, which includes periodic and final evaluation; it is not pure optimizer-step throughput. Peak memory is `torch.cuda.max_memory_allocated`. All parameters were trainable, so tokens per trainable parameter equal tokens per total parameter. Activated parameters are an analytical architecture-level estimate, not measured FLOPs.

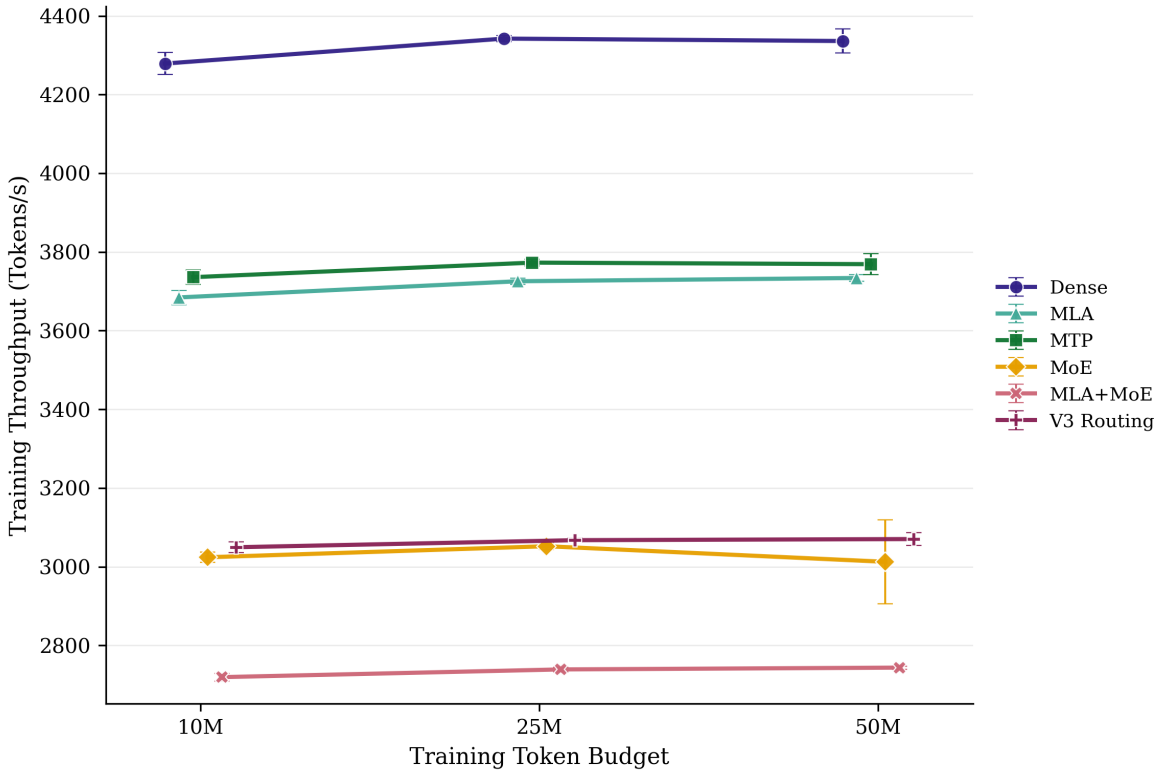


**Figure A.2:** Measured throughput across token budgets. Points are means across ten aligned seeds and bars are 95% Student-$t$ intervals; higher is faster. Throughput is training target tokens per measured end-to-end run second, including periodic and final evaluation, and is not a pure kernel or optimizer-step benchmark. Horizontal offsets are display-only.

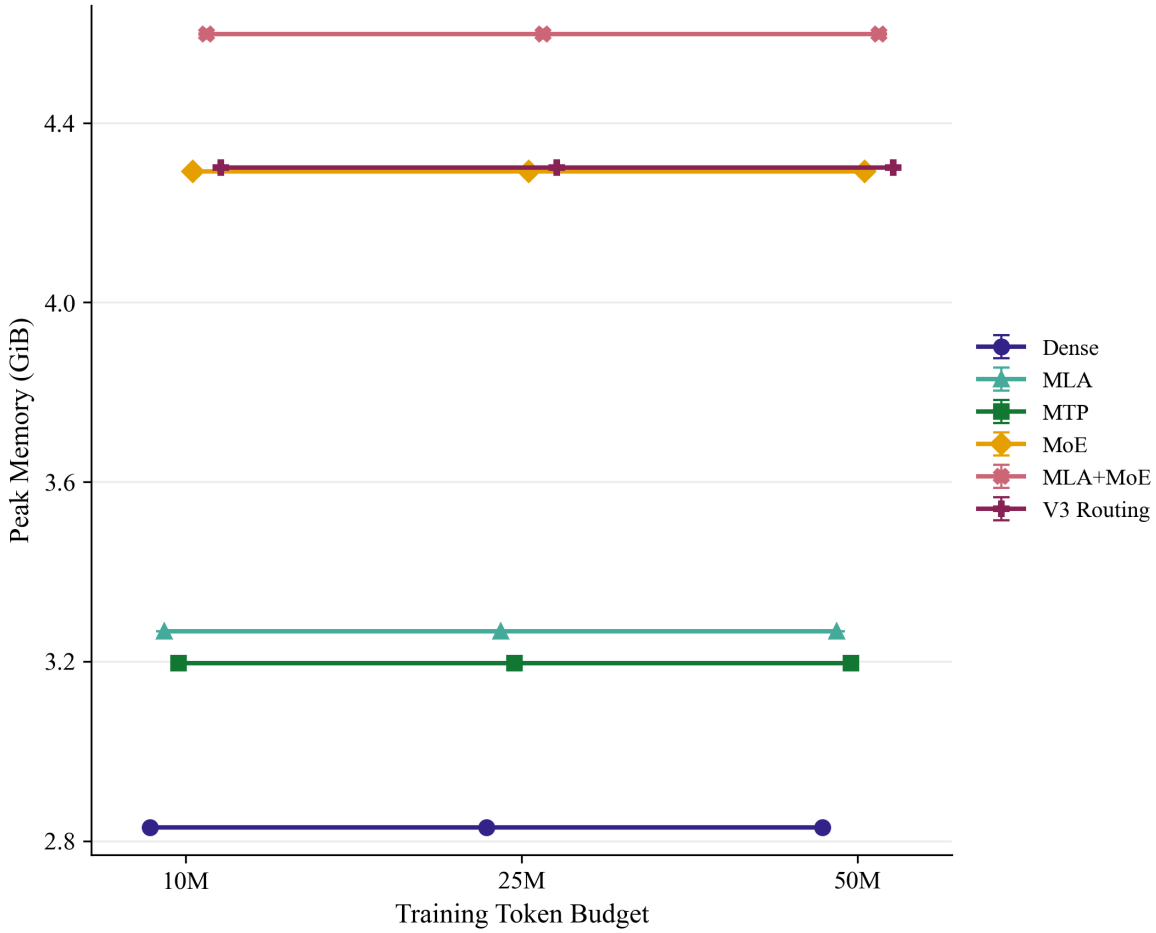


**Figure A.3:** Peak allocated CUDA memory across token budgets. Points are means across ten aligned seeds and bars are 95% Student-$t$ intervals. The metric is `torch.cuda.max_memory_allocated`; it does not represent reserved memory, total device occupancy, or host memory. Horizontal offsets are display-only.

## B. Complete Inferential Results

The main text reports the comparisons needed to answer the research questions. This appendix provides the complete prespecified test-loss family, the paired systems contrasts, the within-model token-budget changes, and supplementary omnibus tests. The aligned seed is the repeated experimental unit throughout.

**Table B.1:** Complete prespecified test-loss contrasts. Differences are Model B minus Model A, so negative values favor Model B. Confidence intervals use 20,000 paired percentile-bootstrap resamples; exact two-sided sign-flip tests use the ten aligned seed differences. The final column applies Holm correction across all 21 primary test-loss contrasts.

| Budget | Contrast (B − A) | Mean diff. | Bootstrap 95% CI | Cohen's $d_z$ | B better | Exact $p$ | Holm $p$ |
|---|---|---|---|---|---|---|---|
| 10M | MLA − Dense | 0.290 | [0.256, 0.326] | 4.873 | 0/10 | 0.001953 | 0.041016 |
| 10M | MoE − Dense | -0.056 | [-0.094, -0.025] | -0.943 | 9/10 | 0.007812 | 0.093750 |
| 10M | MLA+MoE − Dense | 0.260 | [0.230, 0.285] | 5.572 | 0/10 | 0.001953 | 0.041016 |
| 10M | MLA+MoE − MLA | -0.031 | [-0.061, -0.000] | -0.583 | 7/10 | 0.093750 | 1.000000 |
| 10M | MLA+MoE − MoE | 0.315 | [0.270, 0.363] | 4.002 | 0/10 | 0.001953 | 0.041016 |
| 10M | V3 Routing − MoE | 0.025 | [-0.015, 0.062] | 0.378 | 3/10 | 0.253906 | 1.000000 |
| 10M | MTP − Dense | -0.010 | [-0.037, 0.014] | -0.232 | 5/10 | 0.498047 | 1.000000 |
| 25M | MLA − Dense | 0.277 | [0.227, 0.328] | 3.208 | 0/10 | 0.001953 | 0.041016 |
| 25M | MoE − Dense | -0.023 | [-0.063, 0.015] | -0.339 | 6/10 | 0.322266 | 1.000000 |
| 25M | MLA+MoE − Dense | 0.258 | [0.204, 0.314] | 2.749 | 0/10 | 0.001953 | 0.041016 |
| 25M | MLA+MoE − MLA | -0.019 | [-0.053, 0.013] | -0.332 | 5/10 | 0.328125 | 1.000000 |
| 25M | MLA+MoE − MoE | 0.281 | [0.251, 0.322] | 4.578 | 0/10 | 0.001953 | 0.041016 |
| 25M | V3 Routing − MoE | 0.004 | [-0.033, 0.040] | 0.059 | 5/10 | 0.882812 | 1.000000 |
| 25M | MTP − Dense | -0.035 | [-0.080, 0.006] | -0.478 | 7/10 | 0.156250 | 1.000000 |
| 50M | MLA − Dense | 0.190 | [0.169, 0.210] | 5.429 | 0/10 | 0.001953 | 0.041016 |
| 50M | MoE − Dense | -0.026 | [-0.061, 0.008] | -0.435 | 6/10 | 0.207031 | 1.000000 |
| 50M | MLA+MoE − Dense | 0.158 | [0.125, 0.193] | 2.692 | 0/10 | 0.001953 | 0.041016 |
| 50M | MLA+MoE − MLA | -0.032 | [-0.068, 0.003] | -0.527 | 8/10 | 0.121094 | 1.000000 |
| 50M | MLA+MoE − MoE | 0.184 | [0.144, 0.226] | 2.636 | 0/10 | 0.001953 | 0.041016 |
| 50M | V3 Routing − MoE | 0.020 | [-0.018, 0.057] | 0.311 | 4/10 | 0.351562 | 1.000000 |
| 50M | MTP − Dense | -0.026 | [-0.062, 0.011] | -0.408 | 6/10 | 0.246094 | 1.000000 |

*Note.* Test loss is the primary inferential outcome. The all-budget Holm family was selected because it controls multiplicity across the complete prespecified test-loss analysis while avoiding a redundant family based on perplexity, which is a monotonic transformation of loss.

**Table B.2:** Planned throughput and peak-memory contrasts. Relative change is $100(B - A)/A$. Positive throughput changes favor Model B; negative memory changes favor Model B. Holm correction is applied separately across the 21 planned contrasts for each systems metric.

| Budget | Contrast (B − A) | Throughput change | Throughput Holm $p$ | Memory change | Memory Holm $p$ |
|---|---|---|---|---|---|
| 10M | MLA − Dense | -13.9% | 0.041016 | +15.4% | 0.041016 |
| 10M | MoE − Dense | -29.3% | 0.041016 | +51.6% | 0.041016 |
| 10M | MLA+MoE − Dense | -36.4% | 0.041016 | +62.4% | 0.041016 |
| 10M | MLA+MoE − MLA | -26.2% | 0.041016 | +40.8% | 0.041016 |
| 10M | MLA+MoE − MoE | -10.1% | 0.041016 | +7.1% | 0.041016 |
| 10M | V3 Routing − MoE | +0.8% | 0.041016 | +0.2% | 0.041016 |
| 10M | MTP − Dense | -12.7% | 0.041016 | +12.9% | 0.041016 |
| 25M | MLA − Dense | -14.2% | 0.041016 | +15.4% | 0.041016 |
| 25M | MoE − Dense | -29.7% | 0.041016 | +51.6% | 0.041016 |
| 25M | MLA+MoE − Dense | -36.9% | 0.041016 | +62.4% | 0.041016 |
| 25M | MLA+MoE − MLA | -26.5% | 0.041016 | +40.8% | 0.041016 |
| 25M | MLA+MoE − MoE | -10.3% | 0.041016 | +7.1% | 0.041016 |
| 25M | V3 Routing − MoE | +0.5% | 0.041016 | +0.2% | 0.041016 |
| 25M | MTP − Dense | -13.1% | 0.041016 | +12.9% | 0.041016 |
| 50M | MLA − Dense | -13.9% | 0.041016 | +15.4% | 0.041016 |
| 50M | MoE − Dense | -30.5% | 0.041016 | +51.6% | 0.041016 |
| 50M | MLA+MoE − Dense | -36.7% | 0.041016 | +62.4% | 0.041016 |
| 50M | MLA+MoE − MLA | -26.5% | 0.041016 | +40.8% | 0.041016 |
| 50M | MLA+MoE − MoE | -8.9% | 0.041016 | +7.1% | 0.041016 |
| 50M | V3 Routing − MoE | +1.9% | 0.095703 | +0.2% | 0.041016 |
| 50M | MTP − Dense | -13.1% | 0.041016 | +12.9% | 0.041016 |

*Note.* Systems contrasts describe this single-GPU implementation and measured run path. They do not estimate production kernel efficiency, distributed communication cost, inference latency, or KV-cache savings. For peak memory, very small but seed-consistent differences can yield small adjusted p-values; practical magnitude should therefore be read alongside statistical significance.

**Table B.3:** Test-loss change across token budgets. Differences are 50M minus 10M, so negative values indicate improvement. Confidence intervals are paired 95% Student-$t$ intervals across the ten aligned seeds; exact $p$-values use a two-sided sign-flip test.

| Model | 10M | 25M | 50M | 50M−10M | 95% CI | Improved | Exact $p$ |
|---|---|---|---|---|---|---|---|
| Dense | 5.19078 | 4.72909 | 4.48224 | -0.709 | [-0.766, -0.651] | 10/10 | 0.001953 |
| MLA | 5.48108 | 5.00626 | 4.67192 | -0.809 | [-0.887, -0.732] | 10/10 | 0.001953 |
| MTP | 5.18046 | 4.69410 | 4.45663 | -0.724 | [-0.787, -0.660] | 10/10 | 0.001953 |
| MoE | 5.13507 | 4.70636 | 4.45648 | -0.679 | [-0.750, -0.607] | 10/10 | 0.001953 |
| MLA+MoE | 5.45040 | 4.98757 | 4.64007 | -0.810 | [-0.862, -0.758] | 10/10 | 0.001953 |
| V3 Routing | 5.16021 | 4.71004 | 4.47626 | -0.684 | [-0.770, -0.598] | 10/10 | 0.001953 |

**Table B.4:** Supplementary global model-effect tests. Both procedures preserve the aligned-seed block structure; permutation $p$-values use 20,000 within-seed model-label permutations.

| Metric | Budget | RM-ANOVA $F(5, 45)$ | Perm. $p$ | Friedman $Q(5)$ | Perm. $p$ |
|---|---|---|---|---|---|
| Validation loss | 10M | 155.842 | 0.000050 | 39.829 | 0.000050 |
| Validation loss | 25M | 186.448 | 0.000050 | 35.600 | 0.000050 |
| Validation loss | 50M | 109.794 | 0.000050 | 38.171 | 0.000050 |
| Throughput | 10M | 18293.844 | 0.000050 | 50.000 | 0.000050 |
| Throughput | 25M | 49652.410 | 0.000050 | 50.000 | 0.000050 |
| Throughput | 50M | 869.492 | 0.000050 | 47.714 | 0.000050 |
| Peak allocated memory | 10M | 1745063.028 | 0.000050 | 50.000 | 0.000050 |
| Peak allocated memory | 25M | 1745063.028 | 0.000050 | 50.000 | 0.000050 |
| Peak allocated memory | 50M | 1745063.028 | 0.000050 | 50.000 | 0.000050 |

*Note.* Perplexity tests are omitted because perplexity is a monotonic transformation of loss and therefore adds no independent ordering or inferential family.

## C. Mechanism Diagnostics

The routing and MTP measurements below are retained as mechanism-specific diagnostics rather than primary outcomes. Their temporal meaning is deliberately narrow. Routing statistics summarize the final evaluated test batch and MTP loss comes from the final optimizer step. They can support interpretation of the terminal model state but cannot establish full training-trajectory stability, expert specialization, or convergence speed.

**Table C.1:** Terminal routing diagnostics for the sparse models. Entries are means with 95% Student-$t$ intervals across ten seeds.

| Budget | Model | Aux. loss [95% CI] | Routing entropy [95% CI] | Load variance [95% CI] |
|---|---|---|---|---|
| 10M | MoE | 0.010390 [0.010330, 0.010450] | 1.563060 [1.544159, 1.581961] | 0.000401 [0.000360, 0.000442] |
| 10M | MLA+MoE | 0.010337 [0.010298, 0.010377] | 1.652640 [1.628225, 1.677056] | 0.000398 [0.000362, 0.000433] |
| 10M | V3 Routing | 0.000000 [0.000000, 0.000000] | 1.423738 [1.402258, 1.445218] | 0.000732 [0.000660, 0.000804] |
| 25M | MoE | 0.010307 [0.010270, 0.010345] | 1.755014 [1.740144, 1.769883] | 0.000398 [0.000359, 0.000437] |
| 25M | MLA+MoE | 0.010252 [0.010221, 0.010284] | 1.770276 [1.750726, 1.789825] | 0.000358 [0.000330, 0.000387] |
| 25M | V3 Routing | 0.000000 [0.000000, 0.000000] | 1.683988 [1.667969, 1.700006] | 0.000667 [0.000596, 0.000738] |
| 50M | MoE | 0.010266 [0.010240, 0.010292] | 1.950950 [1.939090, 1.962809] | 0.000404 [0.000368, 0.000440] |
| 50M | MLA+MoE | 0.010222 [0.010199, 0.010244] | 1.971264 [1.956107, 1.986420] | 0.000408 [0.000363, 0.000454] |
| 50M | V3 Routing | 0.000000 [0.000000, 0.000000] | 1.915790 [1.895052, 1.936527] | 0.000658 [0.000623, 0.000693] |

*Note.* These are latest-forward snapshots from the final evaluated test batch, averaged across the 12 MoE layers and then summarized across seeds. They are not trajectory averages. V3 Routing's auxiliary loss is exactly zero by design. The load-variance values are low in magnitude; the corresponding main-body figure uses a narrowed axis only to make those differences visible.

**Table C.2:** Final optimizer-step MTP auxiliary loss. The value is diagnostic and is not a training trajectory, convergence-rate estimate, or cross-model-comparable objective.

| Budget | Mean MTP loss | 95% CI |
|---|---|---|
| 10M | 5.50872 | [5.32125, 5.69618] |
| 25M | 5.26094 | [5.14579, 5.37610] |
| 50M | 5.14024 | [4.95374, 5.32674] |

## D. Reproducibility and Artifact Lineage

The report is tied to the repository state at commit `8923ccb2536c` (full hash retained in the repository provenance). The post-audit evidence index records 38 analysis artifacts and 22 figures; this document intentionally selects only the subset required by the approved research questions.

### D.1 Canonical artifact chain

**Table D.1:** Canonical artifact lineage used to construct the report.

| Role | Canonical repository artifact |
|---|---|
| Balanced experiment contract | `results/analysis/balanced_10seed_matrix_manifest.csv` |
| Per-run machine-readable record | `results/analysis/balanced_10seed_matrix_summary_flat.csv` |
| Descriptive statistics | `results/analysis/balanced_10seed_matrix_descriptives_full_precision.csv` |
| Global repeated-measures tests | `results/analysis/balanced_10seed_matrix_global_tests_full_precision.csv` |
| Planned paired contrasts | `results/analysis/balanced_10seed_matrix_paired_contrasts_full_precision.csv` |
| Token-budget trends | `results/analysis/balanced_10seed_matrix_budget_trends_full_precision.csv` |
| Mechanism profiles | `results/analysis/balanced_10seed_matrix_mechanism_profiles.csv` |
| Data and tokenizer provenance | `results/analysis/final_data_tokenizer_provenance.json` |
| Hashes and evidence classification | `results/analysis/balanced_10seed_matrix_evidence_index.json` |

The statistical tables in this report were generated from the full-precision CSV artifacts and rounded only for presentation. The analysis scripts use 20,000 permutation draws for the global tests and 20,000 paired bootstrap resamples for the planned contrasts. Exact sign-flip probabilities are retained where their discrete resolution is relevant. The primary inferential family is the 21 planned test-loss contrasts across all budgets, adjusted by Holm's procedure.

### D.2 Configuration and execution boundaries

The model and training configurations are represented by the tracked YAML files referenced by the manifest. The final corpus is FineWeb-Edu `sample-10BT`, capped at 50,000 training, 1,000 validation, and 1,000 test documents. The retained provenance record identifies the actual streaming shuffle seed as 1337. The byte-level BPE tokenizer has vocabulary size 10,000 and was trained only on retained training text.

The runs were seeded and aligned but are not guaranteed to be bitwise identical across machines, software versions, or CUDA kernels. The implementation does not provide complete interruption-level checkpoint resumption. Throughput and memory values are therefore reproducible measurements of the retained local environment, not portable hardware constants.

### D.3 Regeneration and audit scope

The repository contains dedicated scripts to build and audit the manifest, extract the flat run record, compute descriptives, conduct paired and global inference, construct token-budget trends and mechanism profiles, verify data/tokenizer provenance, produce the figures, and rebuild the evidence index. The post-audit state reports 257 passing tests together with successful formatting, linting, typing, dependency, whitespace, provenance, metric-hash, and figure-hash checks. These audit records establish internal artifact consistency; they do not remove the substantive limitations described in Section 5, including the deferred RoPE correction and the absence of a corrected full rerun.

### D.4 Presentation policy

Loss and perplexity are displayed to five decimal places in tables, effect sizes and parameter ratios to three, $p$-values to six, throughput to one decimal place, and memory to two decimal places in GiB. All calculations use full precision before presentation rounding.

## References


[1] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901, 2020.

[2] Damai Dai et al. DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models, 2024.

[3] DeepSeek-AI. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model, 2024.

[4] DeepSeek-AI. DeepSeek-V3 technical report, 2024.

[5] DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning, 2025.

[6] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1): 1–26, 1979. doi: 10.1214/aos/1176344552.

[7] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

[8] Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Roziere, David Lopez-Paz, and Gabriel Synnaeve. Better & faster large language models via multi-token prediction, 2024.

[9] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Oriol Vinyals, Jack W. Rae, and Laurent Sifre. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 30016–30030, 2022.

[10] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

[11] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models, 2020.

[12] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. GShard: Scaling giant models with conditional computation and automatic sharding. In *International Conference on Learning Representations*, 2021.

[13] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.

[14] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[15] Guilherme Penedo et al. The FineWeb datasets: Decanting the web for the finest text data at scale, 2024.

[16] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding, 2021.

[17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.